

STUDENT GUIDE

INFORMATION TECHNOLOGY FACULTY

INTERNET PROGRAMMING 622

Registered with the Department of Higher Education as a Private Higher Education Institution under the Higher Education Act, 1997. Registration Certificate No. 2000/HE07/008

FACULTY OF INFORMATION TECHNOLOGY

LEARNER GUIDE

MODULE: INTERNET PROGRAMMING 622

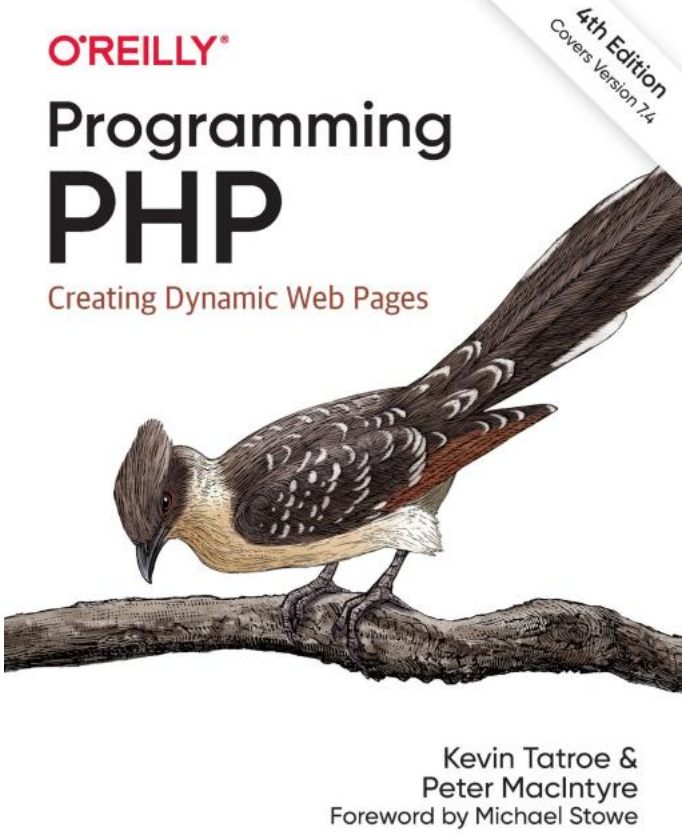
PREPARED ON BEHALF OF

RICHFIELD GRADUATE INSTITUTE OF TECHNOLOGY (PTY) LTD

RICHFIELD GRADUATE INSTITUTE OF TECHNOLOGY (PTY) LTD

Registration Number: 2000/000757/07

All rights reserved; no part of this publication may be reproduced in any form or by any means, including photocopying machines, without the written permission of the Institution.

 O'REILLY® Programming PHP Creating Dynamic Web Pages 4th Edition Covers Version 7.4 Kevin Tatroe & Peter MacIntyre Foreword by Michael Stowe	Programming PHP: Creating Dynamic Web Pages, 4th Edition (2020) by Kevin Tatroe and Peter MacIntyre.
--	---

Contents

Topic 1: PHP and HTML Form.....	9
1.0 Introduction.....	9
1.1 How HTML Forms Work.....	10
1.2 Creating An HTML Form.....	12
1.3 Capturing Form Data With PHP.....	20
1.4 PHP Form Data And Security.....	28
1.5 Handling Empty Form Fields.....	30
1.6 Generating Web Forms With PHP	33
1.7 PHP Form Validation.....	37
Review Questions	40
Topic 2: Session Controls and Cookies	42
2.0 Introduction.....	42
2.1 Cookies.....	43
2.2 Sessions	50
2.3 Alternative to Cookies	54
2.4 Combining Cookies And Sessions	55
Review Questions	57
Topic 3: File System Management.....	59
3.1 Understanding Files And Directories.....	60
3.2 Getting Information on Files.....	61
3.3 Time-Related Properties	62
3.4 Retrieving A Filename from A Path.....	63
3.5 Opening And Closing Files	63
3.6 Opening A File With fopen()	66
3.7 Closing Files With fclose()	68
3.8 Reading and Writing To Files	69
3.9 Reading and Writing Strings Of Characters.....	73
3.10 Testing For The End Of A File.....	77
3.11 Reading One Line At A Time.....	79
3.12 Reading And Writing Entire Files	80
3.13 Random Access To File Data	83

3.14 Copying, Renaming And Deleting Files	85
3.15 Working With Directories	87
3.16 Other Directory Functions	90
3.17 Working With Directory Objects	93
3.18 Telling A File from A Directory	96
Review Questions	99
Topic 4 DATABASES AND STRUCTURED QUERY LANGUAGE [SQL]	102
4.0 Introduction.....	102
4.1 Database Architecture	103
4.2 Embedded Databases.....	104
4.3 Client-Server Databases.....	104
4.4 Database Models	104
4.5 Simple Databases.....	105
4.6 Relational Databases	105
4.7 Understanding Relational Databases	106
4.8 Using Indexes and Keys	107
4.9 Introducing SQL Statements	108
4.10 Creating A New Database	110
4.11 Creating A Table	111
4.12 Adding Data to A Table With The INSERT Statement.....	113
4.13 Reading Data from A Table with The SELECT And SELECT ... Where Clauses.....	116
4.14 Updating Data in A Table Using The UPDATE and UPDATE ... WHERE Clauses	119
4.15 Deleting Data From A Table	120
4.16 Deleting Tables from Databases Using The DROP TABLE Statement	121
4.17 Connecting To MySQL From PHP	123
4.18 Making A Connection	124
4.19 Handling Errors	125
4.20 Reading Data.....	128
Review questions.....	132
Addendum 622 – Installing PHP	134
What is XAMPP?.....	134
Why Do You Need XAMPP?	142

How to run PHP with XAMMP	143
References	146

The Information Technology (IT) qualification at Richfield College is a dynamic and future-focused program designed to equip students with advanced technical, analytical, and problem-solving skills. At the core of the qualification is a commitment to academic excellence, industry alignment, and innovation, fostering graduates who are proficient in addressing modern technological challenges. This qualification strategically integrates theoretical knowledge with practical applications, preparing students for various roles in the IT sector. The IT program is structured to address the growing complexity of the evolving technological landscape.

The Higher Certificate in Information Technology (HCIT) program is a foundational stepping-stone for students who wish to pursue further studies or enter the workforce. Graduates of this program are well-prepared to articulate to the Diploma in IT (DIT) or the Bachelor of Science in IT (BSc IT) qualifications, providing a seamless transition for those seeking to deepen their knowledge and skills in specialized IT areas. Additionally, the program equips students with the essential competencies for entry-level IT roles such as IT Support Technicians, Junior Web/System Developers, IT Administrators, etc.

The Diploma in Information Technology (DIT) is a comprehensive and practical program designed to build a strong foundation in IT principles while equipping students with the hands-on skills required to meet industry demands. Focused on both theoretical knowledge and applied learning, this qualification prepares students for intermediate-level roles in IT and serves as a stepping stone for further academic progression or specialization. Graduates of this program are well-prepared to articulate to the Bachelor of Science in IT (BSc IT) qualification. The curriculum covers programming, networking, database management, system analysis, etc., ensuring graduates possess the competencies to solve real-world IT challenges effectively.

The Bachelor of Science in IT (BSc IT) program is structured to address the growing complexity of the evolving technological landscape. Through carefully curated modules, students gain a deep understanding of software development, database management, cloud computing, cybersecurity, IT management, artificial intelligence, machine learning, networking, etc. Graduates of this program are well-prepared to articulate to the Bachelor of Science Honours in IT qualification. The curriculum is designed to bridge the gap between academic learning and real-world applications, thus fostering innovation and an entrepreneurial mindset. Students are encouraged to participate in research and practical learning.

The programming focus within the IT qualification exemplifies academic innovation and professional alignment. By integrating a diverse range of programming languages with practical application, the curriculum prepares students to excel in the rapidly evolving tech industry. The program aligns with industry courses from globally recognized leading tech giants, such as Oracle, AWS, IBM, etc., ensuring that graduates possess the credentials to validate their expertise in software development and cloud-based technologies. This blend of foundational knowledge, practical experience, and industry-standard courses prepares students for immediate employment and establishes a strong basis for long-term career advancement in software development.

Internet Programming 622 is a specialized module designed to equip students with advanced web development skills using server-side scripting and database integration. This module focuses on PHP and MySQL, enabling students to build dynamic, data-driven web applications. The module emphasizes hands-on learning, where students engage in real-world projects that simulate industry scenarios. By the end of Internet Programming 622, students will be proficient in developing interactive web applications with a strong backend foundation, preparing them for roles such as Web Developers, Full-Stack Developers, and Software Engineers.

Topic 1: PHP and HTML Form



LEARNING OUTCOMES

After reading this Section of the guide, the learner should be able to:

- Explain the importance of web standards for functionality and accessibility across devices.
- Describe universal design principles that make web content accessible to all users.
- Identify the benefits of accessibility and credible resources for web development.
- Outline the evolution of the Internet and HTML's role in information sharing.
- Understand ethical considerations like data privacy and content accuracy.
- Describe how browsers and servers interact to display web content.
- Recognize key protocols (HTTP, HTTPS, FTP) for secure data exchange.
- Define URIs, URLs, and the DNS system for resource access.

1.0 Introduction

One of the most common ways to receive input from the user of a Web application is via an HTML form. You have probably filled in many HTML forms yourself. Common examples include contact forms that let you email a site owner; order forms that let you order products from an online store; and Web - based email systems that let you send and receive email messages using your Web browser. In this chapter, we highlight the importance of building interactive Web forms with PHP. We look at:

- Creating HTML forms

- Writing PHP scripts to capture the data sent from your forms
- Some of the security issues surrounding form data
- How to handle empty form fields, as well as form fields that send more than one value at once
- Using PHP scripts to generate Web forms, giving your forms a lot of flexibility
- Creating forms with built-in error checking
- How to use hidden form fields to create a user - friendly three - stage registration form
- Creating forms that allow users to upload files
- How to use page redirection to make your forms smoother and safer to use
- Creating form validation.

Once you have worked through this chapter, you'll be able to use Web forms to make your PHP scripts much more useful and flexible.

1.1 How HTML Forms Work

Understanding HTML Forms Before Using PHP

Before diving into PHP, it's important to understand how an **HTML form** is structured. An HTML form, or **web form**, is simply a collection of HTML elements embedded within a

webpage. By adding different types of elements, you can create form fields such as **text fields, dropdown menus, checkboxes, radio buttons, and submit buttons**.

All web forms begin with an opening `<form>` tag and end with a closing `</form>` tag:

```
<form action="myscript.php" method="post">  
  
    <!-- Form contents go here -->  
  
</form>
```

The second line in this example is an **HTML comment**. Everything between `<!--` and `-->` is ignored by the web browser and is only visible in the source code.

Key Attributes of the `<form>` Tag

The `<form>` tag includes two important attributes:

1. **`action`** – Specifies where to send the form data when the user submits the form. This should be either:

- An **absolute URL** (e.g., `http://www.example.com/myscript.php`)
- A **relative URL** (e.g., `myscript.php`, `/myscript.php`, or `../scripts/myscript.php`)

The script at the specified URL should be able to accept and process the form data.

2. ``method`` – Defines how the form data is sent to the server. You can use two methods:

- GET: Suitable for sending **small amounts** of data. It appends form data to the URL, making it easy to bookmark or resubmit.
- POST: Allows sending **larger** amounts of data, making it ideal for handling sensitive information.

Adding Form Fields and Other Elements

Once you have created a basic `<form>` element, you can add various **input fields and controls** to allow user interaction. Additionally, you can include other **HTML elements** such as **headings, paragraphs, and tables** to improve structure and readability.

With this foundation, you can now explore how PHP processes form data.

1.2 Creating An HTML Form

In this example, we create a web form that contains a variety of input fields. This section will guide you through creating different types of form fields while also allowing you to see how they look and function in a web browser. Save the following file as

web_form.html in your document root folder, then open it in your browser to view the form:

```
<!DOCTYPE html>

<html lang="en">

<head>

  <title>An HTML Form</title>

  <link rel="stylesheet" type="text/css" href="common.css" />

</head>

<body>

  <h1>An HTML Form</h1>

  <form action="" method="get">

    <div style="width: 25em;">

      <label for="textField">A text input field</label>

      <input type="text" name="textField" id="textField" value="" />

      <label for="passwordField">A password field</label>

      <input type="password" name="passwordField" id="passwordField" value="" />

      <label for="checkboxField">A checkbox field</label>

      <input type="checkbox" name="checkboxField" id="checkboxField" value="yes" />

    </div>

  </form>

</body>

</html>
```

```
<label for="radioButtonField1">A radio button field</label>
```

```
<input type="radio" name="radioButtonField" id="radioButtonField1"  
value="radio1" />
```

```
<label for="radioButtonField2">Another radio button</label>
```

```
<input type="radio" name="radioButtonField" id="radioButtonField2"  
value="radio2" />
```

```
<label for="submitButton">A submit button</label>
```

```
<input type="submit" name="submitButton" id="submitButton" value="Submit  
Form" />
```

```
<label for="resetButton">A reset button</label>
```

```
<input type="reset" name="resetButton" id="resetButton" value="Reset Form" />
```

```
<label for="fileSelectField">A file select field</label>
```

```
<input type="file" name="fileSelectField" id="fileSelectField" value="" />
```

```
<label for="hiddenField">A hidden field</label>
```

```
<input type="hidden" name="hiddenField" id="hiddenField" value="" />
```

```
<label for="imageField">An image field</label>
```

```
<input type="image" name="imageField" id="imageField" value=""  
src="asterisk.gif" width="23" height="23" />
```

```
<label for="pushButton">A push button</label>
```

```
<input type="button" name="pushButton" id="pushButton" value="Click Me" />
```

```
<label for="pullDownMenu">A pull-down menu</label>
```

```
<select name="pullDownMenu" id="pullDownMenu" size="1">
```

```
<option value="option1">Option 1</option>
```

```
<option value="option2">Option 2</option>
```

```
<option value="option3">Option 3</option>
```

```
</select>
```

```
<label for="listBox">A list box</label>
```

```
<select name="listBox" id="listBox" size="3">
```

```
<option value="option1">Option 1</option>
```

```
<option value="option2">Option 2</option>
```

```
<option value="option3">Option 3</option>
```

```
</select>
```

```

<label for="multiListBox">A multi-select list box</label>

<select name="multiListBox" id="multiListBox" size="3" multiple="multiple">

    <option value="option1">Option 1</option>

    <option value="option2">Option 2</option>

    <option value="option3">Option 3</option>

</select>


<label for="textAreaField">A text area field</label>

    <textarea      name="textAreaField"      id="textAreaField"      rows="4"
cols="50"></textarea>

</div>

</form>

</body>

</html>

```

Figure 1.1 below shows what the form looks like. In this figure, an asterisk image was used for the image field; you will need to use an image of your own. Try clicking each control to see how it functions.

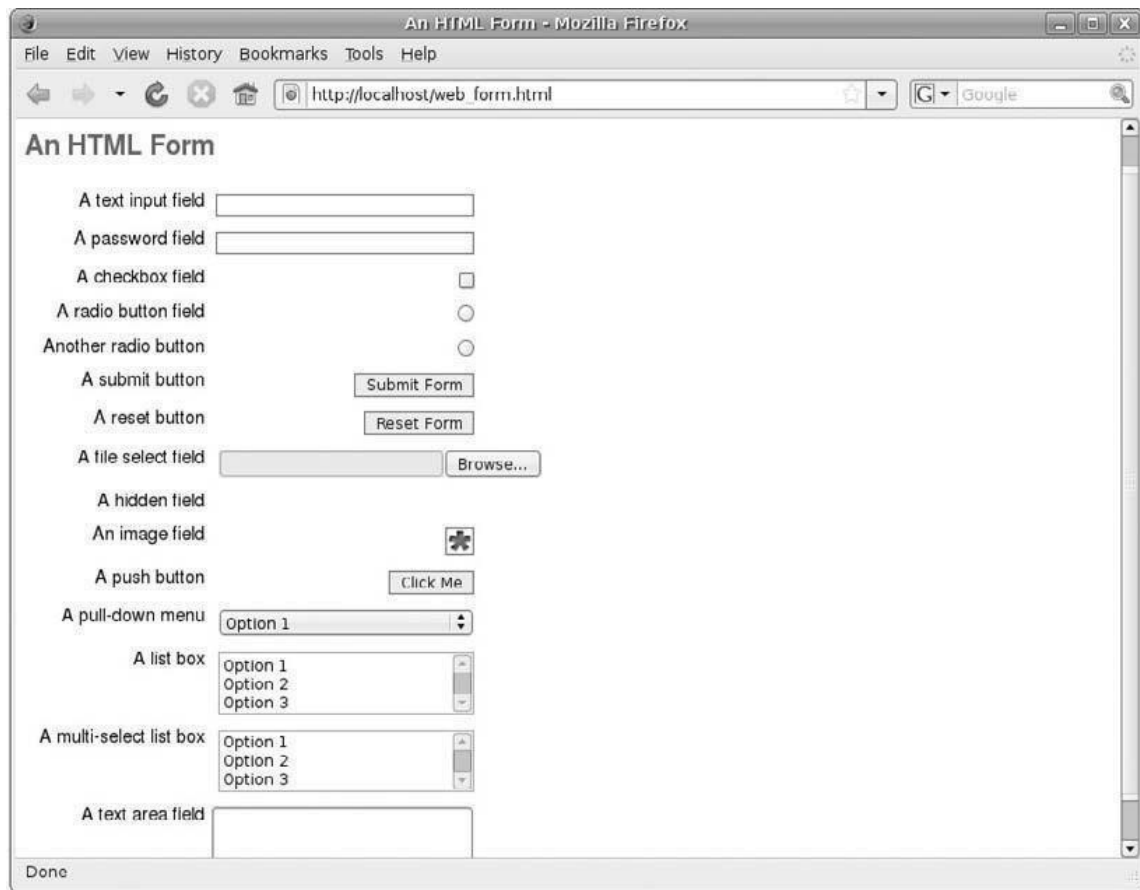


Figure 1.1

This XHTML web page contains the most common types of form controls you are likely to encounter. The form itself is created as follows:

```
<form action="" method="get">
```

Since the form uses the get method, the form field names and values will be sent to the server in the URL. We will introduce the get method shortly. Meanwhile, the empty action attribute tells the browser to send the form data back to the same page (web_form.html). In a real-world scenario, this attribute would contain the URL of a form handler script.

Each form control is then created in sequence. Most controls include a name attribute, which defines the field name that stores the data, and a value attribute, which contains either a fixed value or, for user-input fields, a default value. These field names and values function similarly to keys and values in an associative array.

Additionally, each control is typically associated with a <label> element containing the field's descriptive text, making it easier for users to understand and interact with the form. The for attribute within <label> elements matches the corresponding id attribute of the control.

Once the controls are added, the form is completed with a closing </form> tag.

Try filling in some fields and clicking the Submit Form button. Since the action attribute is empty, the browser sends the form data back to the same page (web_form.html). However, since this page is only an HTML document, it cannot process the data. Soon, you'll learn how to write PHP scripts to handle form submissions.

When the form is submitted, you can see all form data in the browser's address bar, as shown in Figure 1.2. This occurs because the get method sends form data as part of the URL. The data for each field appears as a name/value pair, separated by the ? character:

`http://localhost/web_form.html?textField=Hello&passwordField=secret& ...`

The get method has limitations, as URLs can only contain a limited number of characters (typically up to 1,024 characters). If you need to send larger amounts of data, use the post method instead:

```
<form action="myscript.php" method="post">
```

Unlike get, the post method sends data within the HTTP headers instead of the URL, allowing for larger data transmission. When users refresh the page after submitting a form via post, their browser may display a dialog box asking if they want to resend their form data.

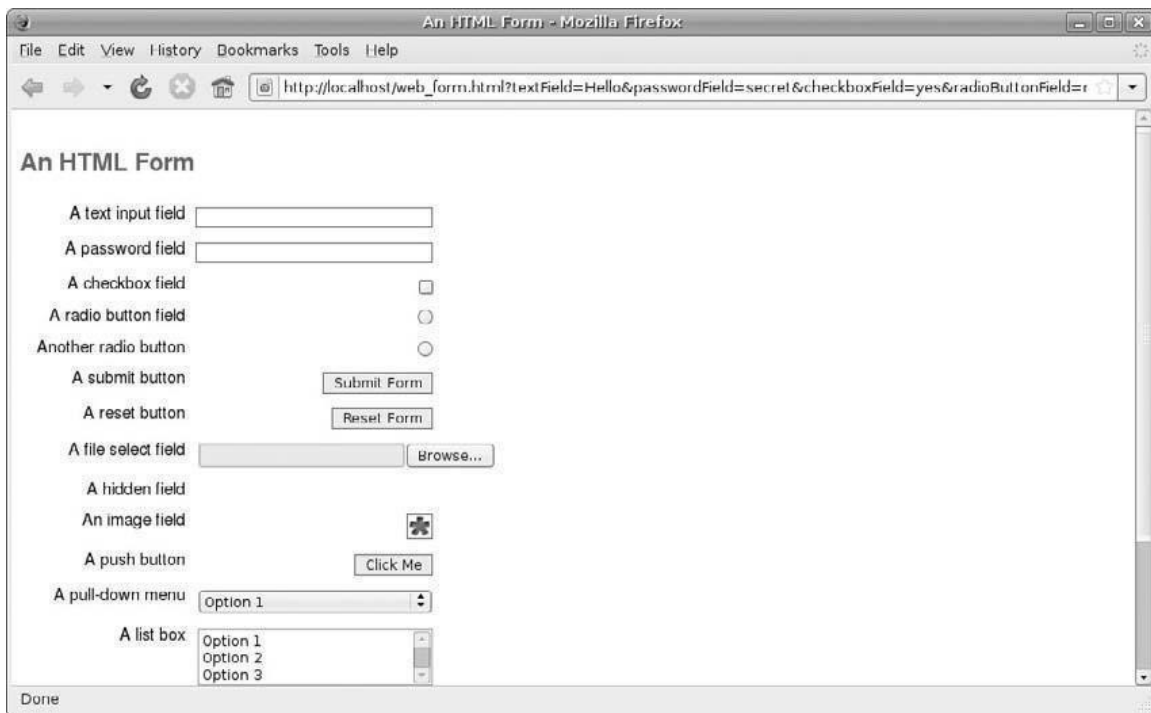


Figure 1.2

1.3 Capturing Form Data With PHP

You now know how to create an HTML form and how data in a form is sent to the server. But how do you write a PHP script to handle that data when it arrives at the server?

First, the form's action attribute needs to specify the URL of the PHP script that will handle the form data. For example:

```
<form action="form_handler.php" method="post">
```

Next, you need to create the `form_handler.php` script. When users submit their forms, their data is sent to the server, and the `form_handler.php` script is executed. The script must read the form data and process it.

To read data from a form, you use superglobal variables. A superglobal is a built-in PHP variable that is available in any scope: at the top level of your script, within a function, or within a class method. Earlier, we discussed the `$GLOBALS` superglobal array, which contains a list of all global variables used in your application. Here, we introduce three new superglobal arrays:

Superglobal Array	Description
<code>\$_GET</code>	Contains a list of all field names and values sent by a form using the GET method.

`$_POST` Contains a list of all field names and values sent by a form using the POST method.

`$_REQUEST` Combines the values of both the `$_GET` and `$_POST` arrays, along with the values of the `$_COOKIE` superglobal array.

Each of these superglobal arrays contains the field names from the form as array keys, with the field values as array values. For example, if you created a form using the GET method with the following field:

```
<input type="text" name="emailAddress" value="" />
```

You could access the value entered by the user using either the `$_GET` or `$_REQUEST` superglobal:

```
$email = $_GET["emailAddress"];
```

```
$email = $_REQUEST["emailAddress"];
```

Example: Creating a User Registration Form

Let's create a simple registration form and a PHP script to process the form data. First, save the following HTML code as `registration.html` in your document root folder:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
    <title>Membership Form</title>
```

```
<link rel="stylesheet" type="text/css" href="common.css" />

</head>

<body>

  <h1>Membership Form</h1>

  <p>Thanks for choosing to join The Widget Club. To register, please fill in your details
  below and click "Send Details".</p>

  <form action="process_registration.php" method="post">

    <div style="width: 30em;">

      <label for="firstName">First name</label>

      <input type="text" name="firstName" id="firstName" value="" />

      <label for="lastName">Last name</label>

      <input type="text" name="lastName" id="lastName" value="" />

      <label for="password1">Choose a password</label>

      <input type="password" name="password1" id="password1" value="" />

      <label for="password2">Retype password</label>

      <input type="password" name="password2" id="password2" value="" />

      <label for="genderMale">Are you male...</label>
```

```
<input type="radio" name="gender" id="genderMale" value="M" />
```

```
<label for="genderFemale">...or female?</label>
```

```
<input type="radio" name="gender" id="genderFemale" value="F" />
```

```
<label for="favoriteWidget">What's your favorite widget?</label>
```

```
<select name="favoriteWidget" id="favoriteWidget" size="1">
```

```
  <option value="superWidget">The SuperWidget</option>
```

```
  <option value="megaWidget">The MegaWidget</option>
```

```
  <option value="wonderWidget">The WonderWidget</option>
```

```
</select>
```

```
<label for="newsletter">Do you want to receive our newsletter?</label>
```

```
<input type="checkbox" name="newsletter" id="newsletter" value="yes" />
```

```
<label for="comments">Any comments?</label>
```

```
<textarea name="comments" id="comments" rows="4" cols="50"></textarea>
```

```
<div style="clear: both;">
```

```
  <input type="submit" name="submitButton" id="submitButton" value="Send  
Details" />
```

```
        <input type="reset" name="resetButton" id="resetButton" value="Reset Form"
style="margin-right: 20px;" />
```

```
    </div>
```

```
</div>
```

```
</form>
```

```
</body>
```

```
</html>
```

Next, create a PHP script named `process_registration.php` to handle the form submission. Save this script in the same folder as `registration.html` and open the `registration.html` URL in your web browser. After filling in the form and clicking the "Send Details" button, the data will be displayed on a new page:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
    <title>Thank You</title>
```

```
    <link rel="stylesheet" type="text/css" href="common.css" />
```

```
</head>
```

```
<body>
```

```
    <h1>Thank You</h1>
```

```
    <p>Thank you for registering. Here is the information you submitted:</p>
```

```
    <dl>
```



```

<dt>First name</dt><dd><?php echo $_POST["firstName"]; ?></dd>

<dt>Last name</dt><dd><?php echo $_POST["lastName"]; ?></dd>

<dt>Password</dt><dd><?php echo $_POST["password1"]; ?></dd>

<dt>Retyped password</dt><dd><?php echo $_POST["password2"]; ?></dd>

<dt>Gender</dt><dd><?php echo $_POST["gender"]; ?></dd>

<dt>Favorite widget</dt><dd><?php echo $_POST["favoriteWidget"]; ?></dd>

<dt>Do you want to receive our newsletter?</dt><dd><?php echo
$_POST["newsletter"]; ?></dd>

<dt>Comments</dt><dd><?php echo $_POST["comments"]; ?></dd>

</dl>

</body>

</html>

```

How It Works

The form is sent using the POST method, so the script accesses the form data from the `$_POST` superglobal array and displays each field value using `echo()`:

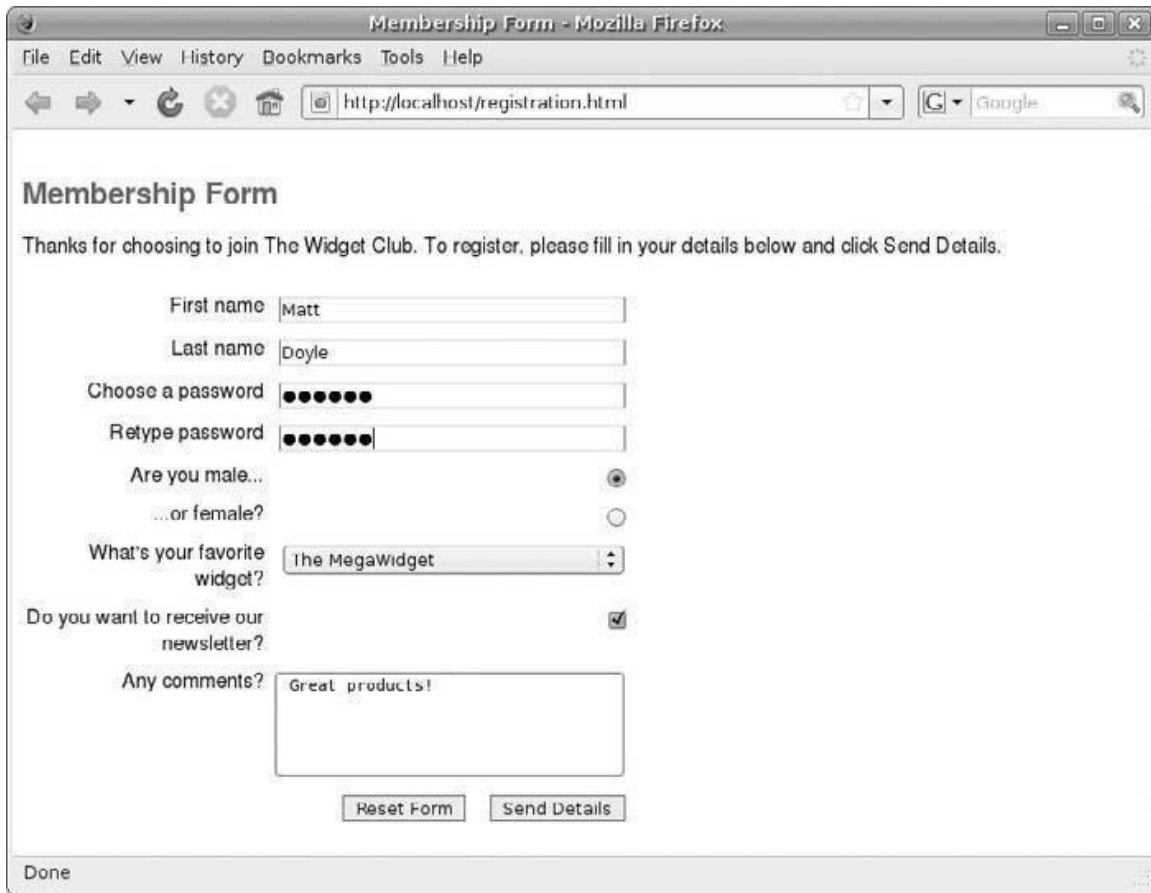
```

<dt>First name</dt><dd><?php echo $_POST["firstName"]; ?></dd>

```

Alternatively, you can use the `$_REQUEST` superglobal, which combines `$_GET` and `$_POST`, to access the form data:

<dt>First name</dt><dd><?php echo \$_REQUEST["firstName"]; ?></dd>



The screenshot shows a Mozilla Firefox browser window titled "Membership Form - Mozilla Firefox". The address bar displays "http://localhost/registration.html". The page content includes a heading "Membership Form" and a message: "Thanks for choosing to join The Widget Club. To register, please fill in your details below and click Send Details." The form fields are as follows: "First name" with the value "Matt"; "Last name" with the value "Doyle"; "Choose a password" and "Retype password" both filled with ten black dots; "Are you male..." with a selected radio button and "...or female?" with an unselected radio button; "What's your favorite widget?" with a dropdown menu showing "The MegaWidget"; "Do you want to receive our newsletter?" with a checked checkbox; and "Any comments?" with a text area containing "Great products!". At the bottom of the form are two buttons: "Reset Form" and "Send Details". The browser's status bar at the bottom left shows "Done".

Figure 1.3 - Form Before Submission

Before submitting the form, it will look something like the form in Figure 1.3, where users fill in their details.

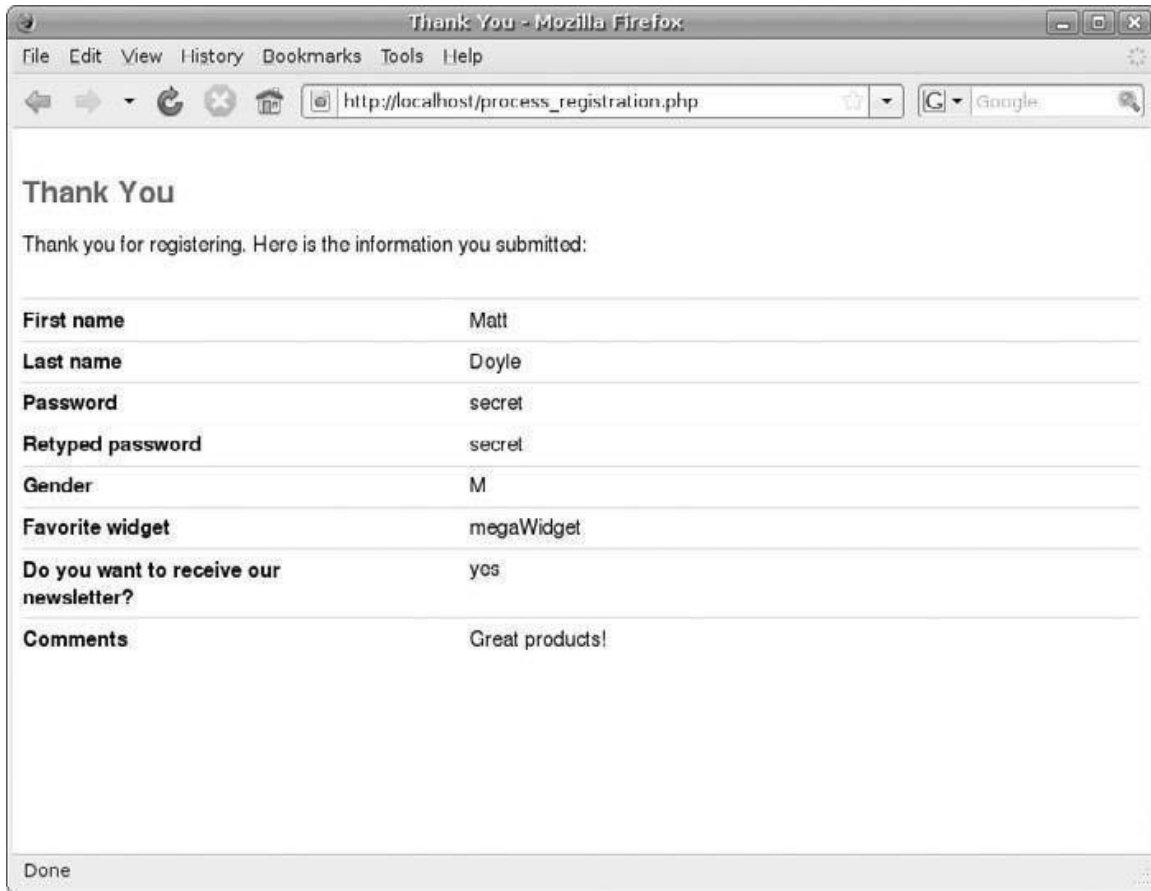


Figure 1.4 - Result After Submission

After submission, the data you entered will be displayed in a list format as shown in Figure 1.4.

Best Practices

Generally, if you know that your form will use either the GET or POST method, it's best to use `$_GET` or `$_POST` specifically rather than `$_REQUEST`. This helps reduce ambiguity, prevent bugs, and avoid potential conflicts with cookies (for example, if there is a cookie with the same name as a form field).

1.4 PHP Form Data And Security

You now know how to create an HTML form and handle data sent to the server. However, it's important to highlight a few security issues with the example script provided earlier.

Security Concerns

Displaying Passwords:

The example script displays the password that the user just entered. In a real-world scenario, you should never display passwords. This could compromise user security. Instead, you could send the user their password via email to remind them of it, but even this should be done cautiously, ensuring that the email is sent securely.

Trusting User Input:

Another major security issue is passing user-entered data — such as the values in `$_GET` and `$_POST` — straight through to a statement like `echo()` or `print()` for displaying in a Web page. You should never trust user input on a public Web site. A malicious user might submit form data intended to exploit vulnerabilities in your website, such as attempting to break into your site or gain access to other users' credentials.

For example, an attacker could submit HTML or JavaScript code that could execute unwanted actions or steal information, known as Cross-Site Scripting (XSS) attacks.

How to Protect Your Site

To avoid these security risks, always validate or filter user input before displaying it on your web page. Here's how you can ensure user data is safe:

Sanitize User Input:

Use built-in PHP functions to sanitize the user input, removing potentially harmful content.

```
$safe_firstName = htmlspecialchars($_POST["firstName"]);
```

The `htmlspecialchars()` function converts special characters to HTML entities, preventing malicious scripts from executing.

Validate User Input:

Make sure that the data is in the correct format. For example, if you're expecting an email address, check that the input matches the expected pattern.

```
if (filter_var($_POST["emailAddress"], FILTER_VALIDATE_EMAIL)) {  
  
    // Proceed with processing the email  
  
} else {  
  
    // Handle invalid email  
  
}
```

Escape Data Before Displaying:

Always escape user input when displaying it to avoid injecting HTML or JavaScript into your page. You can use `htmlspecialchars()` for this purpose.

```
<dt>First name</dt><dd><?php echo htmlspecialchars($_POST["firstName"]); ?></dd>
```

By filtering and validating user input, you reduce the risk of security vulnerabilities and protect your website and users from potential attacks.

Conclusion

Although the previous script is just an example and not designed for real-world use, it's essential to consider security when handling user input. Always sanitize and validate any data submitted by users, especially if it is displayed on the web, to ensure your site is secure from malicious attacks.

1.5 Handling Empty Form Fields

When processing form data in **PHP**, it's important to recognize that users may not fill in all the fields, and some data might not be sent to the server. Depending on form control, when a field is left unfilled or unselected, different behaviors occur.

Behavior of Various Form Controls When Not Filled in or Selected

Form Control	What Happens When It's Not filled-In or Selected
Text input field	The field name is sent, along with an empty value.
Password field	The field name is sent, along with an empty value.
Checkbox field	Nothing is sent at all.

Radio button field	Nothing is sent at all.
Submit button	Nothing is sent at all if the button is not clicked. This can happen if the user presses Enter/Return to submit a form. However, if there is only one submit button in the form, most browsers will still send the button's field name and value.
Reset button	Nothing is ever sent.
File select field	The field name is sent, along with an empty value.
Hidden field	The field name is sent, along with an empty value.
Image field	Same behavior as a submit button.
Push button	Nothing is ever sent.
Pull - down menu	Impossible to select no option, so a value is always sent.
List box	Nothing is sent at all.
Multi - select box	Nothing is sent at all.
Text area field	The field name is sent, along with an empty value.

Why Is This Important?

When a form field doesn't send any data, PHP doesn't create an entry for that field in the ``$_POST`, `$_GET`, or `$_REQUEST` arrays`. Attempting to access such a non-existent element generates a **PHP Notice**, like the following:

PHP Notice: Undefined index: gender in process_registration.php on line 18

This notice typically appears in the server's error log or directly in the browser window (depending on your error reporting settings). Although it doesn't interrupt the script execution, it can clutter logs or display unwanted messages on the page. For example, if ``$_POST["gender"]`` doesn't exist, the following line will still output an empty string:

```
<dt>Gender</dt><dd><?php echo $_POST["gender"] ?></dd>
```

How to Prevent PHP Notices

To avoid PHP notices and ensure that your script handles missing fields gracefully, always check for the presence of a submitted form field before using it. This can be done with PHP functions like ``isset()`` or ``array_key_exists()``.

For example:

```
<dt>Gender</dt><dd><?php if (isset($_POST["gender"])) echo $_POST["gender"];
?></dd>
```


This approach ensures that PHP will only attempt to output the value of ``$_POST["gender"]`` if it exists, preventing the ****undefined index**** notice.

-Research Activity: Multi-Value Fields in PHP

You are encouraged to research how PHP handles ****multi-value form fields****, such as ****checkbox groups****, ****multi-select boxes****, and ****file uploads****. These fields can send multiple values for the same form field name, and understanding how to handle them efficiently is crucial in PHP development.

This topic should be manageable within ****one hour****, and your research can be based on the prescribed book, the internet, or any comparable source.

1.6 Generating Web Forms With PHP

The script allows users to fill out a membership registration form, and it dynamically checks whether the required fields are filled. If fields are missing, the form is redisplayed with error messages. Additionally, it ensures that the previously entered data is retained, allowing users to correct any mistakes without having to re-enter all their information.

Key Components of the Script:

Form Display and Handling:

The form is displayed when the page is loaded, and upon submission, the script checks if the required fields are filled.

If any fields are missing, an error message is displayed, and the form is displayed with the fields that need to be completed highlighted.

Validation of Input:

The script checks for required fields (firstName, lastName, password1, password2, gender).

If any required fields are missing, the form is redisplayed with an error message, and the missing fields are highlighted using the `validateField()` function.

Prefilling Fields:

The script retains the values entered in the form fields by using the `setValue()` function, which echoes the submitted value if it exists. This ensures the form is not reset when there are errors.

Error Handling:

The `validateField()` function adds a CSS class (error) to the labels of fields that were not filled in. This visually indicates to the user which fields need to be completed.

Displaying Thank You Message:

If all required fields are filled, the script displays a thank-you message. This message is shown by the `displayThanks()` function.

Step-by-Step Breakdown of Functions:

`processForm()`: This function processes the form submission. It checks whether all required fields have been filled. If they are not filled, it redisplay the form with error messages.

`displayForm($missingFields)`: This function is responsible for displaying the form. If there are missing fields, it will show an error message, highlight the missing fields, and prefill the values the user already entered.

`validateField($fieldName, $missingFields)`: This function checks if a field is in the `$missingFields` array and, if so, applies the error CSS class to highlight the field.

`setValue($fieldName)`: This function ensures that, if a field was filled in during the previous submission, its value is retained by displaying it in the form input.

`setChecked($fieldName, $fieldValue)`: For radio buttons and checkboxes, this function checks if the field was selected, so the correct options remain selected when the form is redisplayed.

`setSelected($fieldName, $fieldValue)`: Similar to `setChecked`, but for `<select>` options. It ensures that the user's previously selected option is displayed when the form is redisplayed.

displayThanks(): This function displays the thank-you message once the form is correctly filled out.

How the Script Works:

Initial Form Display: When the page is first loaded, the form is displayed with no data. If the user submits the form, the script checks for missing fields and redisplay the form with an error message if needed.

Form Submission and Validation:

If the form is submitted and all required fields are filled, the script shows the thank-you message.

If any required fields are missing, the form is redisplayed with those fields highlighted in red, allowing the user to correct their mistakes.

Form Prefilling: If the user previously entered some data, the script will retain those values and prefill the form, so the user doesn't need to re-enter them after correcting their mistakes.

Example Output:

When you browse the script in your browser:

If you submit an empty form, the missing required fields will be highlighted, and an error message will guide you to fill them in.

If all required fields are completed, a "Thank You" message will be shown.

Figure 1.5

In Figure 1.5 (which you mentioned), you'd see an example of how the form looks when fields are missing or when the form is redisplayed with previously entered values and highlighted missing fields.

Security Consideration:

Although the script works well for this form validation, it doesn't handle security issues like XSS (cross-site scripting) or SQL injection, which should be considered for real-world applications. Always sanitize and validate user input when building forms to prevent malicious users from exploiting vulnerabilities.

1.7 PHP Form Validation

In HTML, forms are used to collect user input/information. But, as far as HTML goes, it just provides the graphical interface and the ability to write on the input fields. However, the aim we collect user information is because we need to process this

information. That usually means saving the inputs to a database so that they can be accessed later for several purposes. In this section, we will not send information to the database, as that requires an active database that is already set up and includes knowledge from SQL language, but we can retrieve information that the user has given us. It is important to have a way to get information, because what you use it for and where you want to show/save it depends. Let's have a look at the HTML form below:

This form is just a regular one, and includes inputs for name, e-mail, age and gender. This information will be the subject of a print when the Submit button is clicked. That just proves that we got the information from the user. For now, let's see what we got and fill in the form:

SAMPLE FORM

Name
E-Mail
Age
☐ Male ☐ Female
Submit

FILLED FORM

Alex Brown
alex.brown@gmail.com
28
☒ Male ☐ Female
Submit

Next, we check each input to make sure the user has written/chosen something, and the input is not empty. We do this using two well-known functions in PHP, the `isset()` and `empty()`. After making sure we have this right, we can also validate fields. In this case, only the name input may need some sort of validation to make sure one simply doesn't write numbers in there, while other inputs are validated from HTML5, in the case of email and age

```

<?php
if (isset($_POST["name"]) && !empty($_POST["name"])) {
    $name = $_POST["name"];
    if (!preg_match("/^[a-zA-Z ]*$/", $name))
        echo "Name: Only letters and whitespace allowed";
    else
        echo "Name: " . $_POST["name"] . " ";
}
if (isset($_POST["email"]) && !empty($_POST["email"])) {
    echo "E-Mail: " . $_POST["email"] . " ";
}
if (isset($_POST["age"]) && !empty($_POST["age"])) {

    echo "Age: " . $_POST["age"] . " ";
}
if (isset($_POST["gender"]) && !empty($_POST["gender"])) {
    echo "Gender: " . $_POST["gender"];
}
?>

```

Validation is a must for all fields when considering real web projects. If the fields have been set, information about each field will be printed. After clicking the "Submit" button, the result we'd have is:

```

Name: Alex Brown
E-Mail: alex.brown@gmail.com
Age: 28
Gender: Male

```

Review Questions

Exercise 1

Create a PHP form that collects a user's name, email, and age. Implement validation to ensure:

- The name and email fields are not empty.
- The email is in a valid format.
- The age is a number between 18 and 60.
- Display appropriate error messages if validation fails.

Exercise 2

Develop a secure PHP login form with username and password fields. Implement the following security measures:

- Prevent SQL injection using prepared statements.
- Sanitize input data to avoid XSS attacks.
- Use `password_hash()` for storing passwords and `password_verify()` for checking passwords.

Exercise 3

Create a PHP form with a multi-select dropdown for selecting favorite programming languages. Ensure that:

- The PHP script can recognize multiple selected values.

- It properly processes and displays the selected values after form submission.

Exercise 4

Create a file upload form in PHP that allows users to upload an image. Implement the following requirements:

- Only allow files with extensions `.jpg`, `.jpeg`, and `.png`.
- Restrict the file size to a maximum of 2MB.
- Move the uploaded file to a `uploads/` directory and display the uploaded image on the page.
- Show appropriate error messages if conditions are not met.

Topic 2: Session Controls and Cookies



LEARNING OUTCOMES

After reading this Section of the guide, the learner should be able to:

- Demonstrate their understanding of saving state in PHP to write powerful and persistent web applications that can remember session information between page views.
- Create cookies, read cookies and delete cookies.
- Understand PHP's session-handling functionalities enabling them to create, sustain and destroy sessions as needed.

2.0 Introduction

Because HTTP is a stateless protocol, most of the PHP scripts you created in previous chapters are very much one-shot affairs. Each time they run, they start with a “clean slate” of variables and other data. This is because each request that a browser makes to a Web server is independent of any previous requests. When a Web server receives a request to

run a PHP script, it loads the script into its memory, runs it, and then removes all traces of it from memory.

However, most of the Web applications you use today have a need to store data between browser requests. For example, a shopping cart needs to remember which items you have added to your cart, and a forum application needs to remember your identity whenever you post a message in the forum. In other words, there is a need to preserve the current state of a user 's interaction with an application from one request to the next. In this chapter, we look at ways to save state, that is to give the web application the ability to store data between browser requests to enable it to remember related or linked actions or requests such as the items that a web user continually adds to his shopping cart. The addition of each incremental item to the cart is a different action but is linked or related to the previous addition as well as to the next. We explore how to use:

- (Query strings to store small amounts of data in the URL)
- Cookies to store larger amounts of data in the browser itself
- Sessions to store even larger amounts of data and store it in a much more secure fashion By using any of these three methods (or a combination of them), you can create persistent Web applications that carry their data from one page view to the next.

Read/activity: You are encouraged to go through your prescribed book, the internet and/or any other comparable source and perform some research pertaining to Query Strings as a “Keeping State” technique in web application development. You may consult your prescribed text between pages 192 and 200 and are strongly urged to consult other sources to augment to the information available here on the subject. The subject is small and should not take more than an hour of your time.

2.1 Cookies

A cookie is a small piece of data that a server can send to a client. On subsequent requests, the client sends this data back to the server, allowing the server to identify the client. Cookies are useful for retaining information across repeated visits by a browser, but they come with a few important considerations.

One key issue is that many browsers allow users to disable cookies, meaning that any application relying on cookies for state maintenance must have an alternative mechanism as a fallback.

The most effective way to manage state in PHP is through its built-in session-tracking system. This system enables you to create persistent variables that remain accessible across different pages of your application and across multiple visits by the same user. PHP's session-tracking system uses cookies (or URLs) behind the scenes to address many of the challenges associated with state management, automatically handling the complexities for you. We'll cover PHP's session system in more detail later in this chapter.

Understanding Cookies

A cookie is essentially a string containing several fields. A server can send one or more cookies to the browser as part of its response headers. Some of these fields specify which pages should send the cookie back to the server, while the value field holds the actual data. This can include information like a unique user identifier, preferences, etc.

To send a cookie to the browser in PHP, use the `setcookie()` function:

```
setcookie(name [, value [, expire [, path [, domain [, secure ]]]]]);
```

This function creates the cookie string based on the given arguments and sends a Cookie header with that string. Since cookies are sent as headers, `setcookie()` must be called before any content (body) is output. Here are the parameters for `setcookie()`:

name: The unique name of the cookie. Multiple cookies with different names and attributes can exist. The name must not contain spaces or semicolons.

value: The arbitrary string associated with the cookie. While there is no explicit limit on the size of a cookie value, the original Netscape specification capped the total size of a cookie (including its name, expiration, and other fields) at 4 KB, so it's best to keep cookie values under 3.5 KB.

expire: The expiration date of the cookie. If not specified, the cookie is stored in memory and expires when the browser closes. The expiration time is given in seconds since midnight on January 1, 1970 (GMT). For example, use `time() + 60 * 60 * 2` to expire the cookie in 2 hours.

path: The browser will only send the cookie for URLs within this path. By default, it is the directory of the current page.

domain: The browser will return the cookie only for URLs within this domain. The default is the server's hostname.

secure: If set to true, the cookie will only be sent over HTTPS connections. By default, it can be sent over unsecured connections.

When the browser sends the cookie back to the server, it can be accessed via the `$_COOKIE` superglobal in PHP. The key is the cookie's name, and the value is the data stored in the cookie. Here's an example that tracks the number of page accesses:

```
$pageAccesses = $_COOKIE['accesses'];  
  
setcookie('accesses', ++$pageAccesses);
```

Cookies can be referenced in a way that any periods (.) in the cookie's name are replaced by underscores. For example, a cookie named `tip.top` is accessed as `$_COOKIE['tip_top']`.

Example: Setting Preferences with Cookies

Here's an example of an HTML form where a user selects background and foreground colors, and their preferences are saved as cookies:

HTML Form:

```
<html>  
  
<head><title>Set Your Preferences</title></head>  
  
<body>  
  
    <form action="prefs.php" method="post">
```

<p>Background:

```
<select name="background">

    <option value="black">Black</option>

    <option value="white">White</option>

    <option value="red">Red</option>

    <option value="blue">Blue</option>

</select><br />
```

Foreground:

```
<select name="foreground">

    <option value="black">Black</option>

    <option value="white">White</option>

    <option value="red">Red</option>

    <option value="blue">Blue</option>

</select>
```

</p>

```
<input type="submit" value="Change Preferences">
```

```
</form>
```

```
</body>
```

```
</html>
```

PHP Script (prefs.php):

```
<html>

<head><title>Preferences Set</title></head>

<body>

<?php

    $colors = array(

        'black' => "#000000",

        'white' => "#ffffff",

        'red' => "#ff0000",

        'blue' => "#0000ff"

    );

    $backgroundName = $_POST['background'];

    $foregroundName = $_POST['foreground'];

    setcookie('bg', $colors[$backgroundName]);

    setcookie('fg', $colors[$foregroundName]);

?>

<p>Thank you. Your preferences have been changed to:<br />
```


Background: <?= \$backgroundName; ?>

Foreground: <?= \$foregroundName; ?></p>

<p>Click here to see the preferences in action.</p>

</body>

</html>

Example Using Color Preferences (prefs_demo.php):

<html>

<head><title>Front Door</title></head>

<?php

 \$backgroundName = \$_COOKIE['bg'];

 \$foregroundName = \$_COOKIE['fg'];

?>

<body bgcolor="<?= \$backgroundName; ?>" text="<?= \$foregroundName; ?>">

 <h1>Welcome to the Store</h1>

 <p>We have many fine products for you to view. Please feel free to browse the aisles and stop an assistant at any time. But remember, you break it you bought it!</p>

 <p>Would you like to change your preferences?</p>

</body>

</html>

Limitations of Using Cookies

There are several limitations and considerations to keep in mind when using cookies:

Cookie Support: Not all clients support or accept cookies, and users can disable them.

Size Limits: The cookie specification limits cookies to 4 KB in total size, with up to 20 cookies allowed per domain, and a maximum of 300 cookies stored on the client side.

Expiration: You don't have control over when browsers expire cookies. If the browser needs space for a new cookie, it may discard older ones, even if they haven't expired yet.

Time Precision: Expiration times depend on the client's system clock, which may not be accurate. Therefore, setting rapid expiration times can lead to unpredictable behavior.

2.2 Sessions

PHP has built-in support for sessions, which simplifies state management by handling cookie manipulation for you. Sessions provide persistent variables that are accessible across different pages and multiple visits to the site. Sessions are ideal for tasks like creating multi-page forms (e.g., shopping carts), storing user authentication data between pages, and saving user preferences.

How Sessions Work

When a user visits your site for the first time, PHP issues a unique session ID. By default, this session ID is stored in a cookie called PHPSESSID. If the user's browser does not support cookies or if cookies are disabled, the session ID is passed through URLs within the site. Each session has an associated data store. You can register variables to be loaded

from this data store at the start of each page and saved back at the end. These variables persist across pages, meaning changes made on one page are visible on others. For instance, an "add to cart" link can add items to a registered array that represents the shopping cart. This array can then be used on other pages to display the cart contents.

Starting and Using Sessions

Sessions start automatically when a script runs. If necessary, a new session ID is generated (which may include sending a cookie to the browser), and any persistent session variables are loaded. You can register a variable in the session by assigning a value to the `$_SESSION[]` array. For example, here's a simple hit counter using sessions:

```
session_start();

$_SESSION['hits'] = $_SESSION['hits'] + 1;

echo "This page has been viewed {$_SESSION['hits']} times.";
```

The `session_start()` function loads registered variables into the `$_SESSION` associative array. The keys are the names of the variables (e.g., `$_SESSION['hits']`). If you need to retrieve the current session ID, you can use the `session_id()` function. To end a session, you call `session_destroy()`. This removes the session data store, but the cookie remains in the browser cache. As a result, the user will have the same session ID on subsequent visits, but no data will be stored.

Example: Setting Preferences with Sessions

Here's an example that shows how to set preferences using sessions instead of cookies.

Setting Preferences with Sessions (prefs_session.php)

```
<html>

<head><title>Preferences Set</title></head>

<body>

<?php

    session_start();

    $colors = array(

        'black' => "#000000",

        'white' => "#ffffff",

        'red' => "#ff0000",

        'blue' => "#0000ff"

    );

    $backgroundName = $_POST['background'];

    $foregroundName = $_POST['foreground'];

    $_SESSION['backgroundName'] = $backgroundName;

    $_SESSION['foregroundName'] = $foregroundName;
```

```
?>
```

```
<p>Thank you. Your preferences have been changed to:<br />
```

```
    Background: <?= $backgroundName; ?><br />
```

```
    Foreground: <?= $foregroundName; ?></p>
```

```
<p>Click <a href="prefs_session_demo.php">here</a> to see the preferences in  
action.</p>
```

```
</body>
```

```
</html>
```

Using Preferences From Sessions (prefs_session_demo.php)

```
<?php
```

```
session_start();
```

```
$backgroundName = $_SESSION['backgroundName'];
```

```
$foregroundName = $_SESSION['foregroundName'];
```

```
?>
```

```
<html>
```

```
<head><title>Front Door</title></head>
```

```
<body bgcolor="<?= $backgroundName; ?>" text="<?= $foregroundName; ?>">
```

```
    <h1>Welcome to the Store</h1>
```

```
<p>We have many fine products for you to view. Please feel free to browse the aisles
and stop an assistant at any time. But remember, you break it, you bought it!</p>
```

```
<p>Would you like to <a href="colors.php">change your preferences?</a></p>

</body>

</html>
```

Session Expiry and Configuration

By default, PHP session cookies expire when the browser is closed, meaning the session does not persist beyond the current session. To change this behavior, you need to adjust the `session.cookie_lifetime` directive in your `php.ini` file to set the cookie's lifetime in seconds

2.3 Alternative to Cookies

By default, PHP passes the session ID between pages using the `PHPSESSID` cookie. However, PHP's session system offers two alternative methods for passing the session ID: form fields and URLs.

Using form fields to pass the session ID can be cumbersome, as it requires every link between pages to be a form submission, which is not practical for most applications.

On the other hand, passing the session ID via URLs is a more seamless approach. PHP can automatically rewrite the HTML, adding the session ID to every relative link. For this to work, PHP must be compiled with the `--enable-trans-id` option. This method does come with a performance trade-off because PHP must parse and rewrite every page. For busy

sites, using cookies may be preferred as they avoid the extra processing overhead caused by page rewriting.

Additionally, passing session IDs through URLs can expose them in the browser's address bar, which increases the risk of man-in-the-middle attacks if proper security measures, like HTTPS, are not in place.

2.4 Combining Cookies And Sessions

By combining cookies with a custom session handler, you can preserve state across visits. PHP's built-in sessions handle temporary state—such as which page a user is on—during a single visit. For more persistent state, such as a unique user ID, you can store that information in a cookie. Using the user's ID, you can then retrieve more permanent data (e.g., display preferences, mailing address) from a persistent store, such as a database.

The following example allows users to select text and background colors, which are stored in a cookie. Any subsequent visits within the next week will send the color preferences stored in the cookie.

Saving State Across Visits (save_state.php)

```
<?php

if ($_POST['bgcolor']) {

    setcookie('bgcolor', $_POST['bgcolor'], time() + (60 * 60 * 24 * 7));

}
```

```

if (isset($_COOKIE['bgcolor'])) {

    $backgroundName = $_COOKIE['bgcolor'];

} else if (isset($_POST['bgcolor'])) {

    $backgroundName = $_POST['bgcolor'];

} else {

    $backgroundName = "gray";

}

?>

<html>

<head><title>Save It</title></head>

<body bgcolor="<?= $backgroundName; ?>">

    <form action="<?php echo $_SERVER['PHP_SELF']; ?>" method="POST">

        <p>Background color:

            <select name="bgcolor">

                <option value="gray">Gray</option>

                <option value="white">White</option>

                <option value="black">Black</option>

                <option value="blue">Blue</option>

                <option value="green">Green</option>

```



```
<option value="red">Red</option>

</select>

</p>

<input type="submit" />

</form>

</body>

</html>
```

Review Questions

Exercise 1: Saving State in PHP (Sessions & Cookies)

Create a simple login system using PHP sessions. The system should:

- Allow a user to log in with a username and password (hardcoded for simplicity).
- Store the username in a session variable after successful login.
- Redirect the user to a welcome page displaying their username.
- Provide a logout option that destroys the session and redirects the user to the login page.

Exercise 2: Creating, Reading, and Deleting Cookies

Write a PHP script that:

- Sets a cookie named `user\_preference` with a value of the user's favorite color.
- Reads and displays the cookie value on the page.
- Provides a button to delete the cookie and refresh the page.

Exercise 3: PHP Session Handling Functionalities

Create a multi-step form (e.g., registration with three steps: Personal Info, Contact Details, and Account Details) that uses PHP sessions to retain data between steps. Ensure that:

- The user can navigate back and forth between steps without losing entered data.
- The final step displays a summary of all entered information before submitting.
- A session is destroyed upon successful submission.

Topic 3: File System Management



LEARNING OUTCOMES

After reading this Section of the guide, the learner should be able to:

- Demonstrate their understanding of files and directories work in PHP.
- Retrieve information on files using various in-built PHP file management and manipulation functions.
- Demonstrate their understanding of how PHP reads and writes to files using various built-in PHP functions.

Introduction

As a server-side programming language, PHP allows you to work with files and directories stored on the Web server. This is very useful, because it means your PHP scripts can store information outside the scripts themselves.

Files are stored in directories on a hard drive, and because they retain their data after the computer is shut down, they are a persistent storage mechanism, instead of temporary storage such as RAM. Directories are a special kind of file made for storing other files. Directories are created hierarchically inside other directories, starting with the root (to level) directory and proceeding down from there.

Files can contain any kind of data and also can contain quite a bit of information about themselves, such as who owns them and when they were created. PHP makes working with the file system easy by including functions that allow you to obtain information about files, as well as open, read from, and write to them.

This chapter is all about the PHP functions for working with the file system. You learn:

- More about files and directories, and how to find out more information about them in your scripts
- How to open and close files, as well as how to read data from, and write data to, files
- The concept of file permissions and how to work with them
- How to copy, move, and delete files
- All about working with directories, including reading their contents, creating them, and deleting them

3.1 Understanding Files And Directories

Everything stored on your hard drive is saved as a file, though we often think of them in terms of files and directories. Files can be program files, data files, directories (which are special files), and system files that help manage the contents of folders. PHP provides functions to work with all types of files, but you'll most often be interacting with text files containing data.

Note: The terms directory and folder are used interchangeably in this guide and refer to the same concept.

File and Directory Basics:

File: A file is simply an ordered sequence of bytes stored on a hard drive or another storage device.

Directory: A directory is a special type of file that contains the names of files and subdirectories (or subfolders) and pointers to their locations on the storage media.

3.2 Getting Information on Files

PHP provides several functions to retrieve useful information about files. For example, before attempting to open a file, you can use the **file_exists()** function to check if it exists:

```
file_exists("/home/chris/myfile.txt");
```

The **file_exists()** function returns `true` if the specified file exists and **false** otherwise.

Similarly, you can use the **filesize()** function to determine the size of a file in bytes:

```
filesize("/home/chris/myfile.txt");
```

This function returns the file size in bytes or `false` if an error occurs (e.g., if the file does not exist).

These functions help ensure that your PHP scripts handle files efficiently and avoid errors when working with the file system.

3.3 Time-Related Properties

In addition to their contents, files have various properties that can provide useful information. The available properties depend on the operating system in which the files are created and modified. For example, on both Windows and UNIX-based systems like Linux, file properties include the time the file was last modified, the time it was last accessed, and the user permissions set on the file.

PHP provides three functions to retrieve time-related information about files:

`fileatime()` — Returns the time at which the file was last accessed, represented as a UNIX timestamp. A file is considered accessed if its contents are read.

`filectime()` — Returns the time at which the file was last changed, also as a UNIX timestamp. A file is considered changed if it was created, written to, or if its permissions were modified.

`filemtime()` — Returns the time at which the file was last modified, again as a UNIX timestamp. The file is considered modified if it was created or its contents were altered.

A UNIX timestamp is an integer value representing the number of seconds since the UNIX epoch (midnight on January 1, 1970).

When working with UNIX timestamps, the `getdate()` function is very useful. It returns an associative array containing the date and time information present in each timestamp. This array includes values like the year, month, day of the month, and more. For example, you can set a variable `$myDate` to the result of `getdate()`, and then access the month component with `$myDate["month"]`.

3.4 Retrieving A Filename from A Path

It is often very useful to separate a filename from its directory path, and the `basename()` function in PHP allows you to do exactly that. This function takes a complete file path and returns only the filename.

For example, the following code assigns `index.html` to the variable `$filename`:

```
$filename = basename("/home/james/docs/index.html");
```

If you provide a directory path instead of a file path, the function will return the rightmost directory name. Here's an example that assigns the value `docs` to `$dir`:

```
$dir = basename("/home/james/docs");
```

Essentially, `basename()` retrieves the last string segment after the rightmost slash (`/`).

If you don't want the file extension (or suffix), you can strip it off by providing the suffix as a second argument to the `basename()` function. For example, the following code assigns `myfile` to `$filename`, removing the `.doc` extension:

```
$filename = basename("/home/james/docs/myfile.doc", ".doc");
```

3.5 Opening And Closing Files

When working with files in PHP, opening a file is the first step, and this is typically done using functions like `fopen()`. A **file handle** is essentially a pointer that represents the open file, allowing your script to interact with it. The file handle is used in subsequent operations (e.g., reading, writing, or closing the file).

Opening a File:

To open a file, you use the `fopen()` function. This function requires two parameters:

1. **File path** – The path to the file you want to open.
2. **Mode** – The mode in which the file should be opened (read, write, append, etc.).

Here are some common modes:

- **r** – Open a file for reading. The file must exist.
- **w** – Open a file for writing. If the file doesn't exist, it will be created. If it exists, its content will be erased.
- **a** – Open a file for writing. If the file doesn't exist, it will be created. If it exists, data will be written at the end of the file.
- **r+** – Open a file for reading and writing. The file must exist.
- **w+** – Open a file for reading and writing. If the file doesn't exist, it will be created. If it exists, its content will be erased.
- **a+** – Open a file for reading and appending. If the file doesn't exist, it will be created.

Example:

```
$file = fopen("example.txt", "r"); // Opens file for reading
if ($file) {
    echo "File opened successfully!";
} else {
    echo "Unable to open the file.";
}
```

File Handle:

Once the file is opened, the `fopen()` function returns a **file handle**. This handle is a pointer that PHP uses to interact with the file. It is essential when performing operations on the file (e.g., reading, writing, or closing).

Reading from a File:

Once the file is opened, you can read its contents using functions like `fread()`, `fgets()`, or `fgetcsv()`.

For example, using `fgets()` to read a file line by line:

```
$file = fopen("example.txt", "r");
if ($file) {
    while (($line = fgets($file)) !== false) {
        echo $line;
    }
    fclose($file); // Close the file when done
} else {
    echo "Unable to open the file.";
}
```

Writing to a File:

You can write to a file using the `fwrite()` function. Make sure the file is opened in a write mode (`w`, `w+`, `a`, or `a+`).

```
$file = fopen("example.txt", "w");
if ($file) {
    fwrite($file, "Hello, World!");
    fclose($file); // Always close the file after writing
} else {
    echo "Unable to open the file.";
}
```

Closing a File:

After you are finished interacting with the file, you should always close it to release the file handle and free system resources. This is done with the `fclose()` function.

`fclose($file); // Closes the file handle`

Summary:

- **fopen()**: Opens a file and returns a file handle.
- **fread()**, **fgets()**, **fwrite()**: Used to read from or write to the file.
- **fclose()**: Closes the file and releases the file handle.

By using file handles, you can efficiently manage the reading and writing of data to files within your PHP script. Remember to always close the file after you're done to avoid memory leaks and potential issues with file locking.

3.6 Opening A File With `fopen()`

Here's an updated and refined version of the text you provided, with clearer explanations and improved flow:

The `fopen()` function in PHP is used to open a file and return a file handle associated with that file. This file handle acts as a pointer, allowing you to read from or write to the file.

Syntax:

```
$handle = fopen("filename", "mode");
```

- **First Argument** (filename): Specifies the path of the file you want to open. This can be:
 - **A simple filename** (e.g., "data.txt"), which tells PHP to look in the current directory.
 - **A relative path** (e.g., "../data.txt"), pointing to the file in a specific subdirectory.
 - **An absolute path** (e.g., "/myfiles/data.txt"), specifying the full path to the file.
 - **A URL** to open a file on a remote server (e.g., `http://www.example.com/index.html` or

ftp://ftp.example.com/pub/index.txt). Note that **remote files can only be opened for reading**—you cannot write to them.

- **Second Argument (mode):** Specifies the mode in which the file is opened. It defines how the file will be used (read, write, append, etc.). Here are the most common modes:
 - "r": Open the file for **reading only**. The file pointer is placed at the beginning of the file.
 - "r+": Open the file for **reading and writing**. The file pointer is placed at the beginning of the file.
 - "w": Open the file for **writing only**. If the file exists, its content will be erased; if it doesn't exist, PHP will attempt to create it.
 - "w+": Open the file for **reading and writing**. Any existing content is erased, and if the file doesn't exist, PHP will attempt to create it.
 - "a": Open the file for **appending only**. Data is written to the end of the file. If the file doesn't exist, PHP will attempt to create it.
 - "a+": Open the file for **reading and appending**. If the file doesn't exist, PHP will attempt to create it.

File Pointer:

PHP uses an internal file pointer to track where the next read or write operation should occur. The pointer's position depends on the mode used to open the file.

Binary and Text Modes:

You can append either 'b' (binary) or 't' (text) to the mode string to specify how the file should be treated. This is especially useful when working with files across different operating systems.

- Binary mode ('b') ensures that the file is treated as a binary file, preventing PHP from making any line-ending conversions.
- Text mode ('t') is typically used when working with text files, allowing PHP to convert end-of-line characters between different operating systems.

For example:

```
$handle = fopen("data.txt", "rb"); // Open in binary mode
```

While UNIX-based systems (Linux/macOS) treat text and binary files the same, **Windows** uses a carriage return and line feed (`\r\n`) to represent line endings, which may cause compatibility issues. Binary mode helps avoid these problems by preventing automatic end-of-line character conversion.

Tip: For portability, it's a good practice to use binary mode ('b') and handle end-of-line characters manually with the PHP constant `PHP_EOL`, which represents the appropriate end-of-line character(s) for the current operating system.

Include Path:

If the filename isn't a relative or absolute path (e.g., "data.txt"), PHP will search the **current directory** by default. However, you can also pass `true` as a third argument to `fopen()`, which instructs PHP to search the **include path** for the file.

Error Handling:

If `fopen()` encounters a problem opening the file (e.g., the file doesn't exist or you lack the required permissions), it will return `false` instead of a file handle. Since file operations are prone to errors, it's essential to implement error handling in your code.

Here's an example of error handling:

```
$handle = fopen("./data.txt", "r");  
if (!$handle) {  
    die("Cannot open the file.");  
}
```

Instead of using `die()`, you might prefer throwing an exception or using a custom error-handling routine, depending on your application's needs.

3.7 Closing Files With `fclose()`

Once you are done working with a file in PHP, it is important to **close** the file to free up system resources and ensure that no data remains in memory that could lead to unexpected behavior. This is done using the `fclose()` function.

Syntax:

```
fclose($handle);
```

- **\$handle:** The file handle that was returned by `fopen()` when the file was opened. This handle is used to reference the file during the script's execution.

Although PHP automatically closes open files when a script finishes running, it's good practice to explicitly close the file once you're done with it. This is particularly important in long-running scripts or when handling multiple files, as it:

- Frees up system resources quicker, allowing other processes or scripts to access the file.
- Ensures that the file's data is properly written or flushed to disk, especially when the file is opened for writing.

Here's an example:

```
$handle = fopen("data.txt", "r");  
  
// Perform file operations...  
  
fclose($handle); // Close the file when done
```

By explicitly closing files when they are no longer needed, you help improve the performance and resource management of your application.

3.8 Reading and Writing To Files

PHP provides a variety of functions to interact with files, allowing you to read and write data in different ways depending on your specific needs. Below is a breakdown of the functions you've mentioned, along with examples and use cases:

Reading from a File:

`fread()`

Reads a string of characters from a file. You specify the length of characters to be

read.

Example:

```
$handle = fopen("file.txt", "r");  
  
$content = fread($handle, filesize("file.txt"));  
  
fclose($handle);
```

fgetc()

Reads a single character at a time from the file. This is useful when you want to process the file character by character.

Example:

```
$handle = fopen("file.txt", "r");  
  
$char = fgetc($handle);  
  
fclose($handle);
```

fgets()

Reads a single line from the file, stopping at the end of the line or the length you specify.

Example:

```
$handle = fopen("file.txt", "r");  
  
$line = fgets($handle);  
  
fclose($handle);
```

fgetcsv()

Reads a line of comma-separated values (CSV) from the file. This is useful for reading CSV files.

Example:

```
$handle = fopen("data.csv", "r");  
  
while (($data = fgetcsv($handle)) !== FALSE) {  
    print_r($data); // Process each CSV line  
}  
  
fclose($handle);
```

file()

Reads the entire file into an array, with each line being an element in the array.

Example:

```
$lines = file("file.txt");  
  
print_r($lines); // Outputs array of lines from file
```

file_get_contents()

Reads the entire file into a string. It's simpler than using fread() and is great for smaller files.

Example:

```
$content = file_get_contents("file.txt");  
  
echo $content; // Outputs the file's content
```

fpasssthru()

Outputs the contents of an open file directly to the browser.

Example:

```
$handle = fopen("image.jpg", "r");  
  
header("Content-Type: image/jpeg");  
  
fpasssthru($handle);  
  
fclose($handle);
```

readfile()

Similar to fpassthru(), this function outputs a file's contents directly.

Example:

```
readfile("file.txt"); // Outputs the contents of the file to the browser
```

Writing to a File:**fwrite():**

Writes a string of characters to a file. It's used when you want to write to a file opened in write or append mode.

Example:

```
$handle = fopen("file.txt", "w");  
  
fwrite($handle, "Hello, world!");  
  
fclose($handle);
```

file_put_contents()

Writes an entire string to a file without needing to open it first. It's a simpler alternative to fwrite().

Example:

```
file_put_contents("file.txt", "Hello, world!");
```

File Pointer Functions:

fseek()

Moves the file pointer to a specific location within the file. This is useful for random access to a file.

Example:

```
$handle = fopen("file.txt", "r");  
  
fseek($handle, 10); // Moves pointer to the 10th byte  
  
$content = fread($handle, 5);  
  
fclose($handle);
```

ftell()

Returns the current position of the file pointer.

Example:

```
$handle = fopen("file.txt", "r");  
  
fseek($handle, 10); // Moves pointer to 10th byte  
  
echo ftell($handle); // Outputs current position of file pointer  
  
fclose($handle);
```

rewind()

Resets the file pointer to the beginning of the file.

Example:


```
$handle = fopen("file.txt", "r");  
  
fseek($handle, 10); // Move pointer to 10th byte  
  
rewind($handle); // Rewind to the start of the file  
  
fclose($handle);
```

Checking End of File:

feof()

Checks if the file pointer has reached the end of the file. This function is often used in a loop to read the entire file until the end.

Example:

```
$handle = fopen("file.txt", "r");  
  
while (!feof($handle)) {  
  
    $line = fgets($handle);  
  
    echo $line;  
  
}  
  
fclose($handle);
```

Conclusion:

With these functions, you can effectively work with files in PHP, whether you're reading from or writing to them. Depending on the situation, you can choose the appropriate function to handle different tasks, from reading entire files to processing them line by line or character by character. If you have any more questions or need examples, feel free to ask!

3.9 Reading and Writing Strings Of Characters

The `fread()` function in PHP is used to read a string of characters from a file. It takes two arguments: the file handle and the number of characters to read. The function reads the specified number of characters (or fewer if the end of the file is reached) and returns them as a string. Here's an example:

```
$handle = fopen("data.txt", "r");  
  
$data = fread($handle, 10);
```

This code reads the first ten characters from `data.txt` and assigns them to `$data` as a string.

When working with binary files, a character is always one byte long, so reading ten characters will read exactly ten bytes. However, when working with Unicode files, where each character may take up multiple bytes, reading ten characters could result in reading, for example, twenty bytes.

After calling `fread()`, the file pointer moves forward by the number of characters read. So in the example above, after the function runs, the file pointer is positioned ten characters forward from the start of the file. If you call `fread()` again, it will read the next ten characters. If fewer than ten characters remain, `fread()` will read and return as many characters as are left.

For reading one character at a time, you can use the `fgetc()` function. It takes a file handle as its argument and returns a single character from the file, or false when the end of the file is reached:

```
$one_char = fgetc($handle);
```

Note that `fgetc()` can be slow when working with large files. It's generally faster to read several characters at once using `fread()` or other file-reading functions.

To write data to a file, use the `fwrite()` function. It requires two arguments: the file handle and the string to write. `fwrite()` writes the contents of the string to the file and returns the number of characters written (or false if there's an error). Here's an example:

```
$handle = fopen("data.txt", "w");  
  
fwrite($handle, "ABCxyz");
```

The first line opens data.txt for writing, which erases any existing content. If the file doesn't exist, PHP attempts to create it. The second line writes the string "ABCxyz" to the beginning of the file. Similar to fread(), after writing, the file pointer moves to the position after the written string. If you call fwrite() again, it will append the same six characters, so the file will contain "ABCxyzABCxyz".

You can limit the number of characters written by providing a third argument. The function will stop writing once it has written the specified number of characters (or fewer if the string is shorter). For example:

```
fwrite($handle, "abcdefghij", 4);
```

This will write only the first four characters "abcd" to the file.

A common use of file functions in web scripts is for creating hit counters, which track how many times a web page has been visited. Below is an example of a simple hit counter script:

```
<!DOCTYPE html>  
  
<html lang="en">  
  
<head>  
  
    <title>Hit counter</title>  
  
    <link rel="stylesheet" type="text/css" href="common.css" />  
</head>  
  
<body>  
  
    <h1>A simple hit counter</h1>  
  
    <?php  
  
        $counterFile = "./count.dat";
```

```

if (!file_exists($counterFile)) {

    if (!($handle = fopen($counterFile, "w"))) {

        die("Cannot create the counter file.");

    } else {

        fwrite($handle, 0);

        fclose($handle);

    }

}

if (!($handle = fopen($counterFile, "r"))) {

    die("Cannot read the counter file.");

}

$counter = (int) fread($handle, 20);

fclose($handle);

$counter++;

echo "<p>You're visitor No. $counter.</p>";

if (!($handle = fopen($counterFile, "w"))) {

    die("Cannot open the counter file for writing.");

}

```

```
fwrite($handle, $counter);  
  
fclose($handle);  
  
?>  
</body>  
</html>
```

This script keeps track of the number of visitors to a page. If the counter file doesn't exist, it creates it and writes an initial value of 0. Then, on each visit, it reads the current count, increments it, and writes the updated count back to the file.

Save this script as `hit_counter.php` and give it a try. Figure 3.1 shows a sample run.

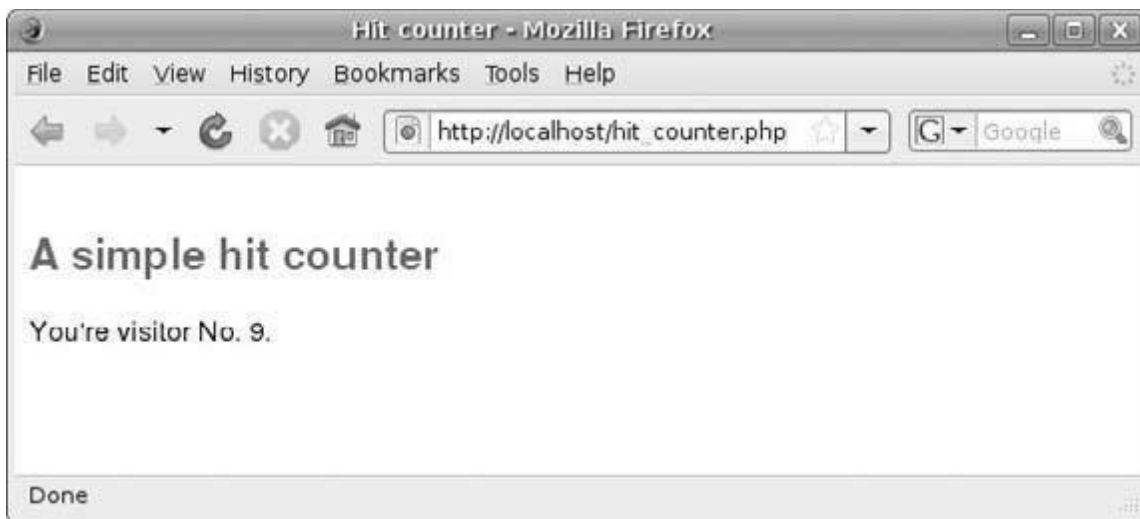


Figure 3.1

3.10 Testing For The End Of A File

The `feof()` function serves a simple purpose: it returns true when the file pointer has reached the end of the file (or if an error occurs) and false otherwise. It takes a single argument — the file handle to test.

Note that `feof()` only returns true after the script has tried to read past the last character in the file. For example:

```
// hello_world.txt contains the characters "Hello, world!"

$handle = fopen("hello_world.txt", "r");

$hello = fread($handle, 13);

echo $hello . "<br />"; // Displays "Hello, world!"

echo feof($handle) . "<br />"; // Displays "" (false)


$five_more_chars = fread($handle, 5);

echo $five_more_chars . "<br />"; // Displays "" (or possibly a newline)

echo feof($handle) . "<br />"; // Displays "1" (true)


fclose($handle);
```

In this example, `feof()` returns false immediately after reading "Hello, world!", because the file pointer is still within the file. After trying to read 5 more characters (which are not available), `feof()` then returns true, indicating the end of the file has been reached.

`feof()` is especially useful when combined with `fread()` or `fgetc()` inside a while loop when you don't know how long the file is. Here's an example of reading a file in chunks:

```
// hello_world.txt contains the characters "Hello, world!"

$handle = fopen("hello_world.txt", "r");

$text = "";

while (!feof($handle)) {

    $text .= fread($handle, 3); // Read 3 characters at a time

}
```

```
echo $text . "<br />"; // Displays "Hello, world!"
```

```
fclose($handle);
```

In this example, `feof()` helps to control the loop, ensuring that the script reads the file in 3-character chunks until it reaches the end.

3.11 Reading One Line At A Time

Reading text from a file one line at a time is often useful, especially when dealing with data files or configuration files that contain one chunk of information per line, such as data records or settings.

To read a line from an open file, use the `fgets()` function. Pass in the file handle, and `fgets()` will read from the current file pointer to the end of the current line. It returns the read characters as a string, including any end-of-line characters (such as `\n` or `\r\n`). If an error occurs, or the end of the file is reached, it returns false.

You can also limit the number of characters read by passing an integer as the second argument. In this case, `fgets()` will stop reading when it reaches that number of characters minus one, unless it encounters the end of the line first. This can be helpful when reading large files that might not contain line breaks.

Here's an example that uses `fgets()` to read and display a three-line text file, one line at a time. The while loop continues until `fgets()` returns false (which happens when the end of the file is reached):

```
/* milton.txt contains:
```

```
The mind is its own place, and in itself
```

```
Can make a Heav'n of Hell, a Hell of Heav'n.
```

```
What matter where, if I be still the same,
```

```
*/
```

```

$handle = fopen("milton.txt", "r");

$lineNumber = 1;

while ($line = fgets($handle)) {
    echo $lineNumber++ . ": $line<br />";
}

```

```
fclose($handle);
```

The code produces the following output:

```

1: The mind is its own place, and in itself
2: Can make a Heav'n of Hell, a Hell of Heav'n.
3: What matter where, if I be still the same,

```

In this example, each line from the milton.txt file is read and displayed with its corresponding line number. The loop continues until the end of the file is reached.

3.12 Reading And Writing Entire Files

Writing code to read a file line by line, or string by string, can be tedious. Fortunately, PHP provides several functions that allow you to access the complete contents of a file in one go, making it easier to work with smaller files. These functions include:

- **file()** — Reads a whole file into an array, without needing to open it.
- **file_get_contents()** and **file_put_contents()** — For reading from and writing to a file without needing to open it.
- **fpassthru()** — Displays the contents of an open file.
- **readfile()** — Displays the contents of a file without needing to open it.

Since these functions read the entire file into memory at once, they are best suited for relatively small files (a few MB at most). For large files, such as a 100MB text file, it's more efficient to use `fread()` or `fgets()` to read and process the file in smaller chunks.

file() Function

The `file()` function reads the contents of a file into an array, with each element containing one line from the file. It takes the filename as its only argument and returns an array with the lines of the file. Here's an example:

```
$lines = file("/home/chris/myfile.txt");
```

Each element of the array will contain one line from the file, including the newline character at the end of each line. The function automatically handles opening, reading, and closing the file, so you don't need to manage the file handle manually.

You can also pass flags as the second parameter to modify the behavior of `file()`. Here are some useful flags:

Flag	Description
FILE_USE_INCLUDE_PATH	Search for the file in the include path (see Chapter 20 for more on include paths).
FILE_IGNORE_NEW_LINES	Removes the newline characters from the end of each line in the array.
FILE_SKIP_EMPTY_LINES	Skips empty lines in the file.

You can combine these flags using the bitwise OR operator. For example, the following code reads a file, looking for it in the include path and ignoring any empty lines:

```
$lines = file("myfile.txt", FILE_USE_INCLUDE_PATH | FILE_SKIP_EMPTY_LINES);
```

You can also use `file()` to fetch files from remote servers:

```
$lines = file("http://www.example.com/index.html");
```

```
foreach ($lines as $line) {  
    echo $line . "<br />";  
}
```

file_get_contents() Function

`file_get_contents()` is similar to `file()`, but it returns the entire file contents as a single string rather than an array of lines. The newline characters are included in the string:

```
$fileContents = file_get_contents("myfile.txt");
```

If there's an issue reading the file, `file_get_contents()` will return `false`.

You can also pass the `FILE_USE_INCLUDE_PATH` flag and specify an offset and length to control where the reading starts and how much of the file is read. For example, to read 23 characters from `myfile.txt` starting at character 17:

```
$fileContents = file_get_contents("myfile.txt", null, null, 17, 23);
```

The first two null values are placeholders for the flags and context arguments, which are not used in this case.

file_put_contents() Function

`file_put_contents()` writes a string to a file. If the file exists, it will be overwritten (unless you specify the `FILE_APPEND` flag). It returns the number of characters written, or `false` if there's a problem:

```
$numChars = file_put_contents("myfile.txt", $myString);
```

You can pass flags as the third argument to modify how the function behaves. The available flags are:

Flag	Description
FILE_APPEND	Appends the string to the file if the file already exists, instead of overwriting it.
LOCK_EX	Locks the file before writing to it, preventing other processes from writing to it simultaneously.

fpassthru() and readfile() Functions

Both `fpassthru()` and `readfile()` allow you to display the contents of a file directly in the browser.

- **fpassthru()** requires an open file handle and outputs the file contents from the current file pointer onward. For example:

```
$numChars = fpassthru($handle);
```

- **readfile()** works with a file path and outputs the contents of the file without needing to open it manually:

```
$numChars = readfile("myfile.txt");
```

Both functions return the number of characters read or false if there's an issue. Note that `fpasssthru()` reads from the current position of the file pointer, so if you have already read part of the file, only the remaining portion will be sent to the browser.

You can also make `readfile()` search the include path by passing `true` as the second argument. This function is particularly useful for sending binary files (like images or PDFs) to the browser for display or download.

3.13 Random Access To File Data

When working with files in PHP, the functions we've covered so far (like `fgets()`, `fread()`, etc.) allow you to manipulate data sequentially—reading or writing the file in the same order the content is arranged. However, sometimes you may need to skip around the file's contents. For instance, you might want to search for a specific string in a file, then return to the beginning of the file to search for another string. While this is easy with small files that you can read entirely into memory using `file_get_contents()`, it's not practical for large files. Fortunately, PHP allows you to move the file pointer within an open file, enabling you to read or write from any point.

PHP provides three functions to control the file pointer:

- **fseek()** — Repositions the file pointer to a specified position in the file.
- **rewind()** — Moves the file pointer back to the start of the file.
- **ftell()** — Returns the current position of the file pointer.

fseek() Function

The `fseek()` function moves the file pointer to a specified position. It requires two arguments: the file handle and an integer offset (the number of characters to move from the starting point). Here's an example where the file pointer is moved to the

eighth character of the file (seven characters after the first one), and then the next five characters are displayed:

```
// hello_world.txt contains the characters "Hello, world!"  
  
$handle = fopen("hello_world.txt", "r");  
  
fseek($handle, 7); // Move the pointer to the 8th character (7 characters from the start)  
  
echo fread($handle, 5); // Displays "world"  
  
fclose($handle);
```

You can also specify how the offset is calculated by passing a third optional argument to `fseek()`, using one of the following constants:

- **SEEK_SET** — Sets the pointer to the start of the file plus the specified offset (this is the default).
- **SEEK_CUR** — Sets the pointer to the current position plus the specified offset.
- **SEEK_END** — Sets the pointer to the end of the file plus the specified offset (use with a negative offset to move backward from the end).

Here's an example using `SEEK_CUR` to move the pointer forward by 5 characters from the current position:

```
fseek($handle, 5, SEEK_CUR); // Move 5 characters forward from the current position  
  
fseek() returns 0 if it successfully moves the pointer, or -1 if there's an error. Note that  
fseek() doesn't work with files accessed via remote hosts over HTTP or FTP (e.g., using  
fopen("http://www.example.com")).
```

rewind() Function

If you want to move the file pointer back to the beginning of the file, you can use the `rewind()` function. This function does the same thing as calling `fseek($handle, 0)` but is a more convenient shorthand:

```
rewind($handle); // Moves the pointer to the start of the file
```

ftell() Function

The `ftell()` function returns the current position of the file pointer, in terms of the number of characters from the start of the file. Here's an example:

```
$offset = ftell($handle); // Returns the current position of the file pointer  
echo $offset; // Outputs the current offset
```

Practical Use with fpassthru()

As you've seen, the fpassthru() function outputs the contents of a file from the current file pointer onward. If you've already read part of a file and want to output its entire contents, you can call rewind() first to reset the file pointer to the beginning before calling fpassthru():

```
rewind($handle);  
fpassthru($handle); // Outputs the entire contents from the beginning
```

3.14 Copying, Renaming And Deleting Files

PHP provides several functions to manage files, such as copying, renaming, and deleting files. These operations can be performed using the following functions:

- **copy()** — Copies a file.
- **rename()** — Renames or moves a file.
- **unlink()** — Deletes a file.

copy() Function

The copy() function copies a file from one location to another. It takes two arguments:

1. The path to the source file.
2. The path where the file should be copied to.

The function returns true if the copy was successful and false if there was an issue. Here's an example of how to use copy():

```
copy("./copyme.txt", "./copied.txt");
```

In this example, the file copyme.txt is copied to copied.txt within the same directory.

rename() Function

The `rename()` function allows you to rename or move a file. It works similarly to `copy()`, but it also renames the file or moves it to a new location. You provide two arguments:

1. The current path and filename.
2. The new path and filename.

Here are two examples:

- Renaming a file:
- `rename("./address.dat", "./address.backup");`

This changes the name of the file `address.dat` to `address.backup`.

- Moving a file to a different folder:
- `rename("/home/joe/myfile.txt", "/home/joe/archives/myfile.txt");`

This moves `myfile.txt` from the `joe` folder to the `archives` folder.

unlink() Function

The `unlink()` function is used to delete files from the server. It only requires one argument: the path to the file you want to delete. For example:

```
unlink("./trash.txt");
```

This deletes the `trash.txt` file from the current directory.

Error Handling

All three functions (`copy()`, `rename()`, and `unlink()`) will raise a warning if there's an issue, such as the file or directory not being found. To avoid such warnings, it's a good practice to check if the file or directory exists before performing any operations. You can use the `file_exists()` function to check for the existence of a file or directory:

```
if (file_exists("./myfile.txt")) {  
    copy("./myfile.txt", "./myfile_copy.txt");  
} else {  
    echo "File does not exist!";  
}
```

```
}
```

This ensures that you're only attempting to copy, rename, or delete files that exist, avoiding unnecessary errors.

3.15 Working With Directories

Working with Directories in PHP

PHP provides several functions for managing directories. These functions are somewhat like the file-handling functions, but they work with directories instead of files. Some functions use a **directory handle**, while others take the directory path as a string.

Directory Handle

A directory handle is like a file handle. It's a special variable that refers to an open directory. You can obtain a directory handle using the `opendir()` function:

```
$handle = opendir("/home/james");
```

If there's an issue opening the directory (e.g., if it doesn't exist), `opendir()` returns `false`.

You can close a directory using the `closedir()` function, passing it the directory handle:

```
closedir($handle);
```

Reading Directory Contents

The `readdir()` function allows you to read the contents of a directory. It expects a directory handle and returns the name of the next entry in the directory:

```
$filename = readdir($handle);
```

Each directory contains entries for files, subdirectories, and special entries:

- `.` represents the directory itself.
- `..` represents the parent directory.

PHP maintains an internal pointer for the directory, similar to how a file pointer works. Each call to `readdir()` moves this pointer to the next entry.

Example: Listing the Contents of a Directory

You can use the functions mentioned above to list the contents of a directory. Below is an example script that lists all files and folders inside a specified directory. You can save this script as `dir_list.php` and test it on your web server.

```
<!DOCTYPE html>

<html lang="en">

<head>

    <title>Listing the contents of a directory</title>

    <link rel="stylesheet" type="text/css" href="common.css" />

</head>

<body>

    <h1>Listing the contents of a directory</h1>

    <?php

        $dirPath = "/home/matt/images"; // Change this to your directory path

        if (!( $handle = opendir($dirPath) )) {

            die("Cannot open the directory.");

        }

    ?>

    <p><?php echo $dirPath ?> contains the following files and folders:</p>

    <ul>

    <?php

        while ($file = readdir($handle)) {

            if ($file != "." && $file != "..") {

                echo "<li>$file</li>";

            }

        }

    }
```



```
        closedir($handle);  
    ?>  
    </ul>  
</body>  
</html>
```

How It Works:

1. **opendir()** opens the specified directory (\$dirPath).
2. **readdir()** reads each entry in the directory. It returns the name of each file or folder, including . and ...
3. We filter out . and .. using an if statement to avoid listing the current and parent directory.
4. **closedir()** closes the directory handle once we're done.

Output

This script will output a list of the files and folders in the specified directory (/home/matt/images) as an unordered list ().

Result Example

If the directory contains files like image1.jpg, image2.png, and a subdirectory subfolder, the output might look like this:

/home/matt/images contain the following files and folders:

- image1.jpg
- image2.png
- subfolder

Summary of Functions:

- **opendir()**: Opens a directory and returns a handle.
- **readdir()**: Reads the next entry in a directory.

- **closedir():** Closes the directory handle.

This approach makes it easy to list the contents of directories in PHP, and it's useful for tasks like managing file uploads, generating file lists, or displaying folder contents on a website.

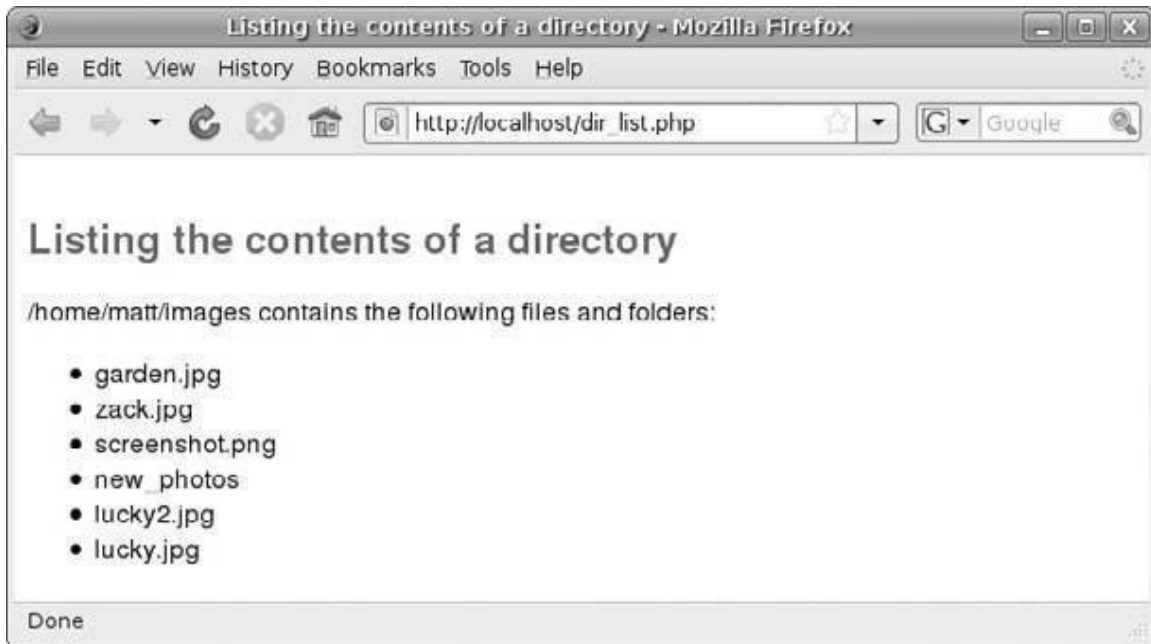


Figure 3.2

3.16 Other Directory Functions

PHP provides several functions for managing directories. These functions allow you to change the current directory, create and remove directories, and navigate through directory entries.

1. rewinddir()

The `rewinddir()` function resets the internal pointer of a directory handle to the first entry in the directory. This is the directory counterpart of the `rewind()` function for files.

Usage:

```
$handle = opendir("/home/matt/images");  
  
while ($file = readdir($handle)) {
```

```
    echo $file . "<br>";  
}
```

```
rewinddir($handle); // Reset the directory pointer to the start
```

This is useful when you want to re-read the entries of a directory from the beginning after iterating through it.

2. chdir()

The `chdir()` function changes the current directory. After calling `chdir()`, relative file paths will be resolved starting from the new current directory.

Usage:

```
chdir("/home/matt/myfolder");
```

```
$handle = fopen("myfile.txt", "r"); // Opens /home/matt/myfolder/myfile.txt
```

You can also use `chdir()` to navigate to a directory and open files in that directory without specifying their full path. It returns true if the directory was successfully changed, or false if there was an error (e.g., directory not found).

3. getcwd()

The `getcwd()` function returns the current directory, which is the directory where PHP will look for files by default.

Usage:

```
chdir("/home/matt/newfolder");
```

```
echo getcwd(); // Displays "/home/matt/newfolder"
```

This function is useful if you want to know the current directory, especially if you're dealing with relative file paths.

4. mkdir()

The `mkdir()` function creates a new directory at the specified path. If the parent directory doesn't exist, the function will fail. You can also specify permissions for the new directory.

Usage:

```
mkdir("/home/matt/newfolder"); // Creates the "newfolder" directory
```

You can also set permissions using the second argument (mode):

```
mkdir("/home/matt/newfolder", 0777); // Creates directory with full permissions
```

5. **rmdir()**

The `rmdir()` function deletes an empty directory. The directory must be empty for `rmdir()` to succeed. If the directory isn't empty, the function will fail and return false.

Usage:

```
rmdir("/home/matt/myfolder"); // Removes "myfolder" directory
```

If the directory isn't empty or PHP doesn't have the required permissions, `rmdir()` will return false.

6. **dirname()**

The `dirname()` function returns the directory portion of a path, excluding the filename. It can be helpful when you want to extract the directory path from a full file path.

Usage:

```
$path = "/home/james/docs/index.html";
```

```
$directoryPath = dirname($path); // Returns "/home/james/docs"
```

```
$filename = basename($path); // Returns "index.html"
```

`dirname()` extracts the directory part, while `basename()` extracts the filename part.

Summary of Directory Functions:

- **rewinddir()**: Resets the internal directory pointer to the first entry.
- **chdir()**: Changes the current directory.
- **getcwd()**: Returns the current working directory.
- **mkdir()**: Creates a new directory.
- **rmdir()**: Deletes an empty directory.

- **dirname()**: Extracts the directory portion from a file path.

These directory manipulation functions make it easier to handle directories, whether you're managing file paths, changing working directories, or creating and removing directories.

3.17 Working With Directory Objects

In PHP, the Directory class offers an object-oriented approach to working with directories. Here's how you can use this class to list directory contents, as well as some of the key properties and methods it provides.

Creating and Using a Directory Object

1. **Creating a Directory Object:** You can create a Directory object using the `dir()` function. This function takes the path to the directory you want to work with as an argument.
2. `$dir = dir("/home/james/docs");`
3. **Directory Properties:** Once the Directory object is created, it has two useful properties:
 - **handle:** This is the directory handle, which can be used with functions like `readdir()`, `rewinddir()`, and `closedir()`.
 - **path:** This is the path to the directory.

Example of using these properties:

```
echo $dir->handle . "<br />"; // Displays the directory handle
```

```
echo $dir->path . "<br />"; // Displays "/home/james/docs"
```

Directory Methods

The Directory class provides three main methods:

1. **read()**: This method is equivalent to `readdir()`, and it reads the next entry in the directory.
2. **rewind()**: This method is equivalent to `rewinddir()`, and it resets the directory pointer to the beginning of the directory entries.
3. **close()**: This method is equivalent to `closedir()`, and it closes the directory handle.

Example: Listing Directory Contents with Directory Object

Here's an example of how to use the Directory class to list the contents of a directory:

```
<!DOCTYPE html>

<html lang="en">

<head>

    <title>Listing the contents of a directory</title>

    <link rel="stylesheet" type="text/css" href="common.css" />

</head>

<body>

    <h1>Listing the contents of a directory</h1>

    <?php

        $dirPath = "/home/matt/images";

        $dir = dir($dirPath);

        ?>
```

```

<p><?php echo $dirPath ?> contains the following files and folders:</p>

<ul>

    <?php

        // Read through the directory and display all entries except "." and ".."

        while ($file = $dir->read()) {

            if ($file != "." && $file != "..") {

                echo "<li>$file</li>";

            }

        }

        $dir->close(); // Close the directory handle

    ?>

</ul>

</body>

</html>

```

Explanation of the Code:

- **Creating the Directory Object:** The `dir()` function is called with the directory path `/home/matt/images`, and it returns a Directory object.
- **Reading the Directory:** The `read()` method is used to read the entries in the directory. It returns the next entry in the directory each time it's called.
- **Filtering Entries:** In the loop, entries `"."` and `".."` are skipped because they represent the current and parent directories, respectively.

- **Closing the Directory:** After reading the entries, the `close()` method is called to close the directory handle.

Summary of the Directory Class Methods:

- **read():** Equivalent to `readdir()`. It reads the next entry in the directory.
- **rewind():** Equivalent to `rewinddir()`. It resets the directory pointer to the start of the list of entries.
- **close():** Equivalent to `closedir()`. It closes the directory handle.

3.18 Telling A File from A Directory

In PHP, the `is_dir()` and `is_file()` functions are useful when you need to determine whether a particular path refers to a directory or a regular file. This is particularly important when you're dealing with file systems that might contain both types of items, such as in a recursive directory traversal or when you want to display only the files in a folder.

Functions to Test File Types:

1. **is_dir():** This function checks whether the given path is a directory.
 - Returns true if the path is a directory.
 - Returns false if it's not a directory (including if it's a regular file or doesn't exist).
2. **is_file():** This function checks whether the given path is a regular file.
 - Returns true if the path is a regular file.
 - Returns false if it's not a regular file (such as a directory or a symlink).

Example Code:

Here's a simple example of using both functions to check if a given filename is a directory, a regular file, or neither:

```
$filename = "myfile";

if (is_dir($filename)) {
    echo "$filename is a directory.";
} elseif (is_file($filename)) {
    echo "$filename is a file.";
} else {
    echo "$filename is neither a directory nor a file.";
}
```

Explanation:

- **is_dir(\$filename):** Checks if myfile is a directory.
- **is_file(\$filename):** If it's not a directory, checks if myfile is a regular file.
- **else:** If neither condition is true, it concludes that the filename is neither a directory nor a regular file (it could be something else like a symlink, a special file, or the file doesn't exist).

Use Case in Directory Traversal:

This is especially useful when traversing directories recursively. For example, when you need to process files in subdirectories, you might want to check whether an entry is a file or a directory before processing it:

```
$directory = "/path/to/directory";

$handle = opendir($directory);

while ($entry = readdir($handle)) {
    if ($entry != "." && $entry != "..") {
```

```
$fullPath = $directory . "/" . $entry;

if (is_dir($fullPath)) {

    echo "$entry is a directory.<br>";

    // You can process the directory (e.g., open it, recurse into it)

} elseif (is_file($fullPath)) {

    echo "$entry is a file.<br>";

    // You can process the file

}

}

}

}

closedir($handle);
```

Summary:

- **is_dir()** checks if the given path is a directory.
- **is_file()** checks if the given path is a regular file.
- Both functions return true or false, helping you make decisions in your code about how to handle files and directories.

Review Questions

Practical Task 1: Demonstrating File and Directory Handling in PHP

1. **Task:** Create a PHP script that checks if a file exists. If it exists, display its size and last modified date. If it doesn't exist, create a new file with some content.
 - Check for the existence of a file named example.txt.
 - If it exists, display the file's size and last modified date.
 - If it doesn't exist, create example.txt and write the text "Hello, this is a new file!" to it.

Practical Task 2: Retrieving Directory Information

2. **Task:** Create a PHP script that lists all files in a directory.
 - Create a directory named files_dir (if it doesn't exist) and populate it with some text files (e.g., file1.txt, file2.txt, file3.txt).
 - List all files in the files_dir directory.
 - Display whether the directory is empty or contains files.

Practical Task 3: Reading and Writing to Files

3. **Task:** Write a PHP script that reads data from a file and then appends additional content.
 - Create a file called data.txt with some initial content (e.g., "Initial data").
 - Read the content of data.txt line by line.

- Append new content ("This is new data added.") to the file.

Practical Task 4: File Error Handling

4. **Task:** Create a PHP script that safely opens a file for reading and writes an error message if the file cannot be opened.
 - Try to open a file called nonexistent.txt for reading.
 - Use error handling (e.g., if statement) to check if the file was successfully opened.
 - If the file does not exist or cannot be opened, display a custom error message.

Practical Task 5: Counting Visits to a Page Using a Hit Counter

5. **Task:** Implement a simple hit counter that stores the number of page visits in a file.
 - Create a file called counter.txt that holds the initial value of 0 (indicating zero visits).
 - Each time the page is visited, read the current value, increment it, and write the new value back to the file.
 - Display the visitor number (e.g., "You are visitor #5.") on the page.

Practical Task 6: Working with CSV Files

6. **Task:** Create a PHP script to read data from a CSV file and display the contents in an HTML table.

- Create a data.csv file with sample comma-separated values (e.g., name,age,city).
- Open the CSV file and read its content line by line.
- Display each row of the CSV file in an HTML table.

Practical Task 7: Reading and Writing Binary Data

7. **Task:** Write a PHP script that reads and writes binary data to a file.

- Open a binary file (e.g., an image or a small binary file).
- Read its content and display the length of the binary data.
- Write some new binary data into a different file.

Topic 4 DATABASES AND STRUCTURED QUERY LANGUAGE [SQL]



LEARNING OUTCOMES

After reading this Section of the guide, the learner should be able to:

- Demonstrate their understanding what databases and the available database models are laying particular emphasis on what a Relational Database System is.
- Write and use complex SQL statements to query a database for the required output.
- Manipulate tables in a database.
- INSERT, UPDATE and DELETE records in any table within a database

4.0 Introduction

In the previous chapter, you learned how to use external files in PHP to store and retrieve data. While files are useful in many cases, they have limitations as data storage solutions. For instance, if you need to filter or sort the data from a file, you must write custom PHP code to handle these tasks. This becomes tedious and inefficient, especially when dealing with large data sets, such as hundreds of thousands of user records. When this happens, your script could slow down significantly, which is far from ideal when you're building a popular website.

Databases were specifically designed to overcome these challenges. With their ability to organize and manage large amounts of data efficiently, they function like well-managed lending libraries, but with superhero-like capabilities. Instead of searching through

endless rows of files, you can simply enter a word or phrase in a search field or command line, and the information you need appears almost instantly.

In this chapter, you will explore how databases work and learn how to integrate them into your PHP applications. You will focus on the essential operations of selecting, inserting, updating, and deleting data. The goal is to give you a solid foundation for working with databases in PHP. Specifically, this chapter will cover:

- The advantages of using databases over files for data storage, particularly in terms of performance and flexibility.
- An introduction to some of the most popular databases, including MySQL, and how they differ from one another.
- The concept of relational databases and how data is organized within them.
- How to configure MySQL, a widely used and free database system, to work with PHP.

By the end of this chapter, you'll have a basic understanding of how to use databases in your PHP applications, setting you up for more advanced work in future projects.

4.1 Database Architecture

Before you get going, you need to settle on a particular database with which to experiment, and that means first deciding on the type of database architecture you are going to use. Broadly speaking, you have two main options: embedded and client-server. Let us take a brief look at both.

4.2 Embedded Databases

An embedded database engine, as the name suggests, operates within the application that uses it (in this case, PHP). It runs and stores its data on the same machine as the host application. The key feature of an embedded database is that it is not networked, meaning it can only be accessed by one program at a time. As a result, the data cannot be shared between multiple machines, and each machine would have its own separate version of the data.

4.3 Client-Server Databases

Client-server databases are more powerful and flexible than embedded databases. They are typically designed for use over networks, allowing multiple applications on different machines to simultaneously access and manipulate the same data. In this setup, the database engine acts as a server, providing data to its clients (like how web servers serve pages to web browsers). It can handle requests from virtually anywhere, if there is a network connection and the appropriate client software. However, it's also common for both the server and client to run on the same machine in some cases. This type of database is typically found in larger organizations, where large amounts of data need to be shared across many users and accessed from various locations. The centralized nature of the database makes tasks like administration and backups more manageable. Applications that interact with the database rely on specialized, lightweight client programs to communicate with the server.

4.4 Database Models

As well as considering the architecture of the database system, it is important to think about the database model you want to use. The database model defines how data is

stored, organized, and accessed. There are many different database models in use today, but in this section, we will focus on two common ones: the simple database model and the relational database model.

4.5 Simple Databases

Simple database engines, as the name suggests, are among the easiest to work with. The simple database model is essentially like an associative array of data, where each item is referenced by a unique key. There are no relationships between the data items in the database, meaning the structure is very straightforward. For smaller applications, the simple database model can offer significant advantages. For instance, if your primary need is to quickly look up data based on keys, simple databases can be extremely fast and efficient.

4.6 Relational Databases

Relational databases offer significantly more power and flexibility than simple databases, which is why they are often the preferred choice for many applications. These databases, also known as RDBMSs (Relational Database Management Systems), are designed to manage and store data in a structured way, allowing for relationships between different data sets. This chapter will focus primarily on RDBMSs.

While RDBMSs provide advanced capabilities, they can also be expensive and complex to set up and administer. The three most well-known RDBMSs are Oracle, DB2 (from IBM), and SQL Server (from Microsoft). These systems are feature-rich and capable of handling nearly any data storage or processing needs a modern business could have. However, their complexity and high costs may be excessive for many projects, especially smaller

ones. Fortunately, there are open-source alternatives such as PostgreSQL and MySQL. These relational database systems have become very popular with PHP developers due to their speed, stability, and ease of use. Both PostgreSQL and MySQL can efficiently meet the needs of most small-to-medium-sized projects, and best of all, they are free!

4.7 Understanding Relational Databases

In simple terms, a relational database is a system that organizes data into tables and allows it to be associated or grouped by common attributes. For instance, payroll records can be grouped by employee, department, or date. In a relational database, data is typically arranged into tables, with each table divided into rows and columns.

Each row in a table represents a data record, which is a set of related pieces of information, such as details about a particular person. Each column represents a field, which is a specific type of data that has the same meaning for every record in the table, such as "first name" or "age."

In database terminology, the terms "row" and "record" are often used interchangeably, as are "column" and "field." Here's an example of a database table:

Player Number	Name	PhoneNumber	DatePlayed	Nickname
42	David	555-1234	03/03/04	Dodge
6	Nic	555-3456	03/03/04	Obi-d
2	David	555-6543	03/03/04	Witblitz
14	Mark	555-1213	03/03/04	Greeny
2	David	555-6543	02/25/04	Witblitz
25	Pads	555-9101	02/25/04	Pads
6	Nic	555-3456	02/25/04	Obi-d
7	Nic	555-5678	02/25/04	Nicrot

Suppose a football team manager sets up a database to track the matches in which his players participate. The players are asked to enter their details after each match. After two matches, the manager's table, called matchLog, might look like this:

In this table, each row represents a specific set of data about a player who played on a particular date. Each column contains a specific type of data for each person or date. The names at the top of each column are called field names or column names, which help identify the type of data contained in that column.

4.8 Using Indexes and Keys

Inexperienced database designers often complain about their database engines being slow, and a common reason for this is the lack of indexing. An index is essentially a separate, sorted list of the values in one or more columns of a table, often referred to as keys. These terms are largely interchangeable. You can add indexes to one or more columns either when you create the table or at any point after the table has been created.

To understand why indexing has such a dramatic effect on performance, let's first consider a table without indexes. Such a table behaves similarly to a plain text file, where the database engine must search through the entire table sequentially. Since

rows in a relational database are inserted in an arbitrary order, the engine must scan the whole table to find entries that match your query. This is slow and inefficient, especially when there are only a few matches.

Now, consider a table with an index. Instead of scanning the entire table, the engine can scan the index, which is a sorted list, to find matching values. This scan is much quicker. The index acts as a guide, allowing the engine to quickly locate the relevant rows in the table, and it eliminates the need for a full table scan, improving query performance dramatically.

A primary key is a special type of index used to uniquely identify records and establish relationships between tables, which is the foundation of the relational database model. Each related table in a relational database should have one and only one primary key. It's also possible to create an index or primary key that involves multiple fields instead of just one. In such cases, the combination of values across these indexed fields must be unique for each record.

While indexes significantly boost search and retrieval performance, they come with trade-offs. For example, adding too many indexes can negatively impact the performance of updates and increase the size of the table. Whenever a record is inserted into an indexed table, the database engine must also update the corresponding index or indexes. The more indexes you have, the slower the update process becomes, and the larger the table grows because of the additional index data. Thus, while indexes are incredibly useful for improving read operations, they should be used judiciously to avoid negative impacts on write operations.

4.9 Introducing SQL Statements

To work with databases and tables, you use SQL statements. These statements allow you to interact with the data stored in the tables. Common SQL statements include:

- **SELECT:** Retrieves data from one or more tables.
- **INSERT:** Inserts data into a table.
- **REPLACE:** Replaces data in a table. If the same record exists, it is overwritten with new data.

- **UPDATE:** Updates data in a table.
- **DELETE:** Deletes data from a table.

There are also SQL statements that deal with the structure of databases and tables, rather than manipulating the data within them:

- **CREATE:** Creates a database, table, or index.
- **ALTER:** Modifies the structure of a table.
- **DROP:** Deletes a database or table.

In this chapter, you'll explore most of these statements in detail, but let's first look at the typical form of a MySQL **SELECT** statement, which is used to retrieve records from a table. SQL operations like this are often referred to as *queries*, hence the name *Structured Query Language*:

```
SELECT field1, field2, ..., fieldn FROM table WHERE condition;
```

A statement can span multiple lines for readability, as shown in this simple example:

```
SELECT lastName, firstName  
  
FROM users  
  
WHERE firstName = 'John';
```

In this query, the **FROM** clause specifies the table to retrieve data from (**users**), and the **WHERE** clause filters the data based on the condition that the **firstName** is 'John'. The output might look like this if there are matching records:

```
Simpleton John  
  
Smith John  
  
Thomas John
```

The values returned from the query are known as the *result set*. You can loop through all the rows in the result set within your PHP script. If the query finds no rows matching the condition, it will return NULL instead. It's also important to note that SQL statements like **DELETE** or **INSERT** do not return a result set; they return an indication of how many rows were affected.

4.10 Creating A New Database

Creating a new MySQL database is straightforward. To begin, open the MySQL command-line tool and enter the following command in the shell prompt on Windows:

```
mysql -u root -p
```

After pressing Enter, you will be prompted to enter the **root** password. Once you've entered the correct password and pressed Enter again, you should see the MySQL prompt:

```
mysql >
```

To create a new database, you use the CREATE DATABASE command. For example, to create a database named **mydatabase**, type the following command:

```
CREATE DATABASE mydatabase;
```

Press Enter, and MySQL will create the new database. You can verify that the database has been created by listing all the databases on the system with the following command:

```
SHOW DATABASES;
```

You should see an output like this:

```
+-----+  
| Database |
```

```
+-----+
| information_schema |
| mydatabase      |
| mysql          |
+-----+
```

3 rows in set (0.00 sec)

As you can see, there are now three databases: **information_schema** and **mysql**, which are system databases used by MySQL, and **mydatabase**, the database you just created.

Remember, it's essential to end each command or statement with a semicolon before pressing Enter in MySQL.

4.11 Creating A Table

To begin with, we'll create a simple table in the **mydatabase** database. Start by selecting the database with the following command:

```
USE mydatabase;
```

Once you press Enter, you should see:

```
Database changed
```

Now, you can proceed to create your table. Use the following CREATE TABLE command to create a table named **fruit** with three fields: **id** (the primary key), **name** (the name of the fruit), and **color** (the fruit's color). Type the following at the mysql > prompt:

```
CREATE TABLE fruit (
    id SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
    name VARCHAR(30) NOT NULL,
```

```
color VARCHAR(30) NOT NULL,  
  
PRIMARY KEY (id)  
  
);
```

Press Enter at the end of each line. MySQL will display the - > arrows automatically to indicate the continuation of the statement on the next line. Once the table is successfully created, you should see a response like this:

```
Query OK, 0 rows affected (0.06 sec)
```

Now that the table is created, you can check the list of tables in your database by running:

```
SHOW TABLES;
```

The output should look like this:

```
+-----+  
| Tables_in_mydatabase |  
+-----+  
| fruit                |  
+-----+
```

```
1 row in set (0.00 sec)
```

If you ever need to create a regular index (as opposed to a primary key) for a specific field in a table, use the KEY or INDEX keyword. For example, if you wanted to add an index for the **name** field in the **fruit** table, you could use the following command:

```
CREATE TABLE fruit (  
  
    id SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,  
  
    name VARCHAR(30) NOT NULL,  
  
    color VARCHAR(30) NOT NULL,  
  
    PRIMARY KEY (id),  
  
    KEY (name)
```


);

This would help improve lookup performance for the **name** field, especially when dealing with large sets of fruit records.

4.12 Adding Data to A Table With The INSERT Statement

Now let's add some fruit to the **fruit** table using the ``INSERT`` statement. As you know, the basic syntax for the ``INSERT`` statement is:

```
INSERT INTO table VALUES (value1, value2, ...);
```

This inserts values into each field of the table in the order that the fields were defined. Since `**id**` is an ``AUTO_INCREMENT`` field, we don't need to include it in our ``INSERT`` statements — it will be filled automatically.

Here is how to insert three rows into the **fruit** table by providing values for only the **name** and **color** fields:

```
INSERT INTO fruit (name, color) VALUES ('banana', 'yellow');
```

After running this, you should see:

Query OK, 1 row affected (0.06 sec)

Next, insert the second fruit record:

```
INSERT INTO fruit (name, color) VALUES ('tangerine', 'orange');
```

You should see:

Query OK, 1 row affected (0.00 sec)

Finally, insert the third fruit record:

```
INSERT INTO fruit (name, color) VALUES ('plum', 'purple');
```

And you should see:

Query OK, 1 row affected (0.00 sec)

If you want to verify that the rows have been successfully added to the table, run the following query:

```
SELECT * FROM fruit;
```

This should show the three records with automatically generated ****id**** values:

id	name	color
1	banana	yellow
2	tangerine	orange
3	plum	purple

3 rows in set (0.00 sec)

You've now added data to your table successfully!

4.13 Reading Data from A Table with The **SELECT** And **SELECT ... Where** Clauses

The **SELECT** statement in SQL is a powerful tool for retrieving data from a database. You can query tables in various ways to filter, sort, and display the information you need.

Here are a few examples using the **fruit** table you've created:

Retrieve All Data

To retrieve all the data from the **fruit** table, you use:

```
SELECT * FROM fruit;
```

This query returns every row and column in the **fruit** table. The * means "all fields" in the table.

Example result:

```
+----+-----+-----+
| id | name   | color |
+----+-----+-----+
| 1 | banana | yellow |
| 2 | tangerine | orange |
| 3 | plum   | purple |
+----+-----+-----+
```

3 rows in set (0.00 sec)

Retrieve Specific Columns

You can also specify which columns to retrieve. For instance, to get just the **name** and **color** columns:

```
SELECT name, color FROM fruit;
```

This will return only the specified columns:

```
+-----+-----+
| name  | color |
+-----+-----+
| banana | yellow |
| tangerine | orange |
| plum   | purple |
+-----+-----+

3 rows in set (0.00 sec)
```

Filter Data Using the WHERE Clause

To retrieve specific rows, you can add a **WHERE** clause to filter the results based on certain conditions.

For example, to get all records where the **name** is 'banana':

```
SELECT * FROM fruit WHERE name = 'banana';
```

The result will be:

```
+----+-----+-----+
| id | name  | color |
+----+-----+-----+
```

```
| 1 | banana | yellow |
```

```
+----+-----+-----+
```

1 row in set (0.08 sec)

Similarly, you can filter based on other conditions. For instance, to retrieve fruits with an **id** greater than or equal to 2:

```
SELECT * FROM fruit WHERE id >= 2;
```

The result:

```
+----+-----+-----+
```

```
| id | name   | color |
```

```
+----+-----+-----+
```

```
| 2 | tangerine | orange |
```

```
| 3 | plum     | purple |
```

```
+----+-----+-----+
```

2 rows in set (0.06 sec)

Key Takeaways:

- **SELECT * FROM table_name** retrieves all data from the table.
- **SELECT column1, column2** lets you specify which columns to retrieve.
- **WHERE** clause helps filter results based on conditions you define, like equality (=) or greater than (>=).

With these basic queries, you can start retrieving specific data from your database to use in your applications!

4.14 Updating Data in A Table Using The UPDATE and UPDATE ... WHERE Clauses

The **UPDATE** statement in SQL allows you to modify existing data within a table. It is commonly used when you need to change one or more values of specific records in the database. However, it is important to use a **WHERE** clause to specify which records should be updated, as leaving it out will update all rows in the table.

Example of the UPDATE statement:

```
UPDATE fruit
```

```
SET name = 'grapefruit', color = 'yellow'
```

```
WHERE id = 2;
```

This command updates the record where id = 2. It changes the **name** to 'grapefruit' and the **color** to 'yellow'. After executing this statement, the fruit table will look like this:

```
+----+-----+-----+
| id | name   | color |
+----+-----+-----+
| 1 | banana | yellow |
| 2 | grapefruit | yellow |
| 3 | plum   | purple |
+----+-----+-----+
```

This is an important operation, especially in database management where you need to keep your data current and relevant. Be careful when using UPDATE without a WHERE clause as it will update all rows, which could be a costly mistake if not intended.

To get deeper into your research:

1. **Subqueries:** These are queries nested inside other queries. They allow you to run complex queries in one go by selecting data from one table and using it in the main query.
2. **Joins:** These allow you to combine rows from two or more tables based on a related column. Common types include **INNER JOIN**, **LEFT JOIN**, and **RIGHT JOIN**.
3. **Unions:** The **UNION** operator allows you to combine results from multiple **SELECT** queries. It eliminates duplicate rows by default, but you can use **UNION ALL** if you want to include duplicates.

As you go through your reading or research, focus on understanding how and when to use these techniques for powerful database management. These concepts are foundational for more complex database interactions and are widely used in web applications, data analysis, and business intelligence.

4.15 Deleting Data From A Table

The **DELETE** statement in SQL is used to remove data from a table. Just like the **UPDATE** statement, you can use a **WHERE** clause to specify which rows to delete. If no **WHERE** clause is provided, all the rows in the table will be deleted, but the table itself will remain intact.

Example of the **DELETE** statement:

```
DELETE FROM fruit WHERE id = 2;
```

This command will delete the row where `id = 2`. After executing the statement, the fruit table will look like this:

```
+----+-----+-----+
| id | name  | color |
+----+-----+-----+
| 1  | banana | yellow |
| 3  | plum   | purple |
```


+----+-----+-----+

It's important to use caution when using the **DELETE** statement, especially without a **WHERE** clause. If you accidentally omit the **WHERE** clause, it will delete all rows in the table, which could result in the loss of important data.

4.16 Deleting Tables from Databases Using The DROP TABLE Statement

To delete a table entirely from a database, you can use the DROP TABLE statement in MySQL. This command will remove the table and all its data permanently. Similarly, you can delete an entire database with the DROP DATABASE statement.

First, let's see how the DROP TABLE command works:

1. Start by listing all the tables in your database using the SHOW TABLES command:

```
mysql > SHOW TABLES;
```

+-----+

| Tables_in_mydatabase |

+-----+

| fruit |

+-----+

1 row in set (0.00 sec)

2. To delete the fruit table, you can use the DROP TABLE statement:

```
mysql > DROP TABLE fruit;
```

Query OK, 0 rows affected (0.25 sec)

3. To confirm that the table has been deleted, you can run the SHOW TABLES command again:

```
mysql > SHOW TABLES;
```

Empty set (0.00 sec)

As you can see, after running the DROP TABLE statement, the table fruit no longer exists in the database.

The DROP DATABASE statement works in a similar manner, but it removes an entire database along with all its tables and data.

1. To list all the databases in MySQL, use the SHOW DATABASES command:

```
mysql > SHOW DATABASES;
```

```
+-----+
| Database      |
+-----+
| information_schema |
| mydatabase     |
| mysql          |
+-----+
```

3 rows in set (0.40 sec)

2. To delete the mydatabase database, use the DROP DATABASE statement:

```
mysql > DROP DATABASE mydatabase;
```

Query OK, 0 rows affected (0.14 sec)

3. To confirm that the database has been deleted, run the SHOW DATABASES command again:

```
mysql > SHOW DATABASES;
```

```
+-----+
| Database      |
+-----+
| information_schema |
| mysql          |
```

+-----+

2 rows in set (0.00 sec)

The mydatabase database has been removed, leaving only information_schema and mysql databases.

Caution:

Be careful when using the DELETE and DROP commands, as these operations are permanent. Once you delete a table or database, it cannot be undone. Always ensure that you back up your databases regularly and carefully consider any action that could potentially result in data loss. Before performing any destructive operation, such as DROP TABLE or DROP DATABASE, it's a good idea to double-check your backups and make sure you're not accidentally removing important data.

Research Activity:

You are encouraged to read pages 203 to 226 in your prescribed book, along with researching the DELETE statement. This statement is used to remove specific records from a table, unlike DROP TABLE, which removes the entire table. Understanding how to safely and efficiently delete records is important when working with databases.

4.17 Connecting To MySQL From PHP

So far, we've covered the theory behind relational databases and worked directly with MySQL using the MySQL command-line tool. Now, it's time to get your PHP scripts interacting with MySQL. At the time of writing, PHP offers two main ways to connect to MySQL databases:

The first option is **mysqli** (MySQL improved), which is specifically designed for MySQL. It provides the most complete access to MySQL from PHP, offering both procedural (function-oriented) and object-oriented interfaces. While it can feel overwhelming due to the large number of functions and classes available, it's a great choice if you know you'll be working exclusively with MySQL and want to leverage the full power of MySQL in your PHP scripts.

The second option is **PDO** (PHP Data Objects), an object-oriented extension that sits between the MySQL server and the PHP engine. It offers a simple and clean set of classes and methods to interact with MySQL databases. The advantage of PDO is that it's not limited to MySQL; it can also be used with a variety of other database systems, such as PostgreSQL, Oracle, and more. This means you only need to learn one set of classes and methods to create applications that can interact with multiple types of databases.

Choosing between these two extensions often sparks debate among PHP developers, as both have their strengths and weaknesses. For this guide, we'll be using PDO because it's easier and quicker to learn. Once you're comfortable with PDO, you'll find that you can easily transfer those skills to mysqli if needed.

If you've installed PHP and MySQL through Synaptic on Ubuntu, XAMPP on Windows, or MAMP on macOS, you should find that both the mysqli and PDO extensions are already installed and ready to use.

4.18 Making A Connection

Connecting to MySQL Database with Procedural Style in PHP (Latest Version)

```
$servername = "localhost";
```

```
$username = "root";
```

```
$password = "";
```

```
$database = "mydatabase";
```

```
// Create connection
```

```
$conn = mysqli_connect($servername, $username, $password, $database);
```

```
// Check connection

if (!$conn) {

    die("Connection failed: " . mysqli_connect_error());

}

echo "Connected successfully";


// Your database operations go here


// Close the connection

mysqli_close($conn);
```

4.19 Handling Errors

Connecting with Error Handling

```
$servername = "localhost";

$username = "root";

$password = "";

$dbname = "mydatabase";


// Create connection

$conn = mysqli_connect($servername, $username, $password, $dbname);
```

```
// Check connection

if (!$conn) {

    die("Connection failed: " . mysqli_connect_error());

}

echo "Connected successfully";
```

Error Handling for Queries

```
// Example query

$sql = "SELECT * FROM some_table";

$result = mysqli_query($conn, $sql);

// Check if query failed

if (!$result) {

    echo "Query failed: " . mysqli_error($conn);

} else {

    // Process the result here

}

// Close the connection

mysqli_close($conn);
```

Using a Custom Error Handling Function

```
function handleDbError($conn) {  
    echo "Error: " . mysqli_error($conn);  
  
    // Optionally log the error or send an email to the webmaster  
  
    // error_log("Database error: " . mysqli_error($conn), 3,  
    "/var/log/my_app_errors.log");  
}  
  
$servername = "localhost";  
$username = "root";  
$password = "";  
$database = "mydatabase";  
  
// Create connection  
$conn = mysqli_connect($servername, $username, $password, $database);  
  
// Check connection  
if (!$conn) {  
    die("Connection failed: " . mysqli_connect_error());  
}  
  
echo "Connected successfully";
```

```
// Example query

$sql = "SELECT * FROM some_table";

$result = mysqli_query($conn, $sql);


// Check if query failed and handle error
if (!$result) {

    handleDbError($conn);

} else {

    // Process the result

}


// Close the connection

mysqli_close($conn);
```

4.20 Reading Data

Assuming you have a table fruit, you can fetch data from it as follows:

Set up the table in MySQL:

```
CREATE DATABASE mydatabase;

USE mydatabase;
```



```
CREATE TABLE fruit (  
    id SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,  
    name VARCHAR(30) NOT NULL,  
    color VARCHAR(30) NOT NULL,  
    PRIMARY KEY (id)  
);
```

```
INSERT INTO fruit (name, color) VALUES ('banana', 'yellow');
```

```
INSERT INTO fruit (name, color) VALUES ('tangerine', 'orange');
```

```
INSERT INTO fruit (name, color) VALUES ('plum', 'purple');
```

PHP script to display data (get_fruit.php):

```
<!DOCTYPE html>  
  
<html lang="en">  
  
<head>  
    <meta charset="UTF-8">  
    <title>Fruit</title>  
    <link rel="stylesheet" type="text/css" href="common.css">  
</head>  
  
<body>  
    <h1>Fruit</h1>  
  
    <?php  
        $servername = "localhost";  
        $username = "root";
```

```

$password = "";

$database = "mydatabase";


// Create connection

$conn = mysqli_connect($servername, $username, $password, $database);


// Check connection

if (!$conn) {

    die("Connection failed: " . mysqli_connect_error());

}


// SQL query to fetch data

$sql = "SELECT * FROM fruit";

$result = mysqli_query($conn, $sql);


// Check for query error

if (!$result) {

    die("Query failed: " . mysqli_error($conn));

}


// Display results

echo "<ul>";

while ($row = mysqli_fetch_assoc($result)) {

```

```
        echo "<li>A " . htmlspecialchars($row["name"]) . " is " .  
        htmlspecialchars($row["color"]) . "</li>";
```

```
    }
```

```
    echo "</ul>";
```

```
// Close the connection
```

```
mysqli_close($conn);
```

```
?>
```

```
</body>
```

```
</html>
```

Review questions

Exercise 1: RDBMS

Describe what a Relational Database Management System (RDBMS) is and list three examples of popular RDBMS software.

Exercise 2: Writing Complex SQL Queries

Given the following database schema for an E-commerce System:

```
CREATE TABLE Customers (  
  
    CustomerID INT PRIMARY KEY AUTO_INCREMENT,  
  
    Name VARCHAR(100),  
  
    Email VARCHAR(100) UNIQUE,  
  
    Country VARCHAR(50)  
  
);  
  
CREATE TABLE Orders (  
  
    OrderID INT PRIMARY KEY AUTO_INCREMENT,  
  
    CustomerID INT,  
  
    OrderDate DATE,  
  
    TotalAmount DECIMAL(10,2),  
  
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
```

);

Write an SQL query to:

- Retrieve all customers who have placed at least one order.
- List all orders along with the corresponding customer's name.
- Get the total amount spent by each customer (use GROUP BY).

Exercise 3: Manipulating Tables in a Database

Modify the Customers table to:

- Add a new column Phone Number that accepts up to 15 characters.
- Change the Country column to accept up to 100 characters.

Write the necessary SQL statements.

Exercise 4: INSERT, UPDATE, and DELETE Records

Using the Customers and Orders tables above, write SQL statements to:

- Insert a new customer named "John Doe" with an email johndoe@example.com from "USA".
- Update the email of "John Doe" to john.doe@newmail.com.
- Delete all orders associated with the customer "John Doe" before removing them from the Customers table.

Addendum 622 – Installing PHP

What is XAMPP?

The acronym XAMPP stands for cross-platform, Apache, MySQL, PHP, and Perl. XAMPP is a free and open-source web server that allows you to develop, test, and build websites on a local server.

Unlike PHP, XAMPP installation is quite simple and uncomplicated. Search for "XAMPP Download" in your browser or go to their [website](#). You should see the current version of XAMPP for Windows, Linux, and OSX when it opens.

Because I'm using a Windows computer, I simply need to click on the one for Windows, and the download should begin.

XAMPP Installers and Downloads

apachefriends.org/index.html

XAMPP Apache + MariaDB + PHP + Perl

What is XAMPP?

XAMPP is the most popular PHP development environment

XAMPP is a completely free, easy to install Apache distribution containing MariaDB, PHP, and Perl. The XAMPP open source package has been set up to be incredibly easy to install and to use.



Download
Click here for other versions

 XAMPP for Windows 8.1.6 (PHP 8.1.6)	 XAMPP for Linux 8.1.6 (PHP 8.1.6)	 XAMPP for OS X 8.1.6 (PHP 8.1.6)
---	--	--

New XAMPP release 7.4.29 ,

Hi Apache Friends!
We just released a new version of XAMPP. You can download these new installers at <http://www.apachefriends.org/download.html>.
These installers include the next components:

Cookie Settings

Click the XAMPP for Windows if you are using Windows

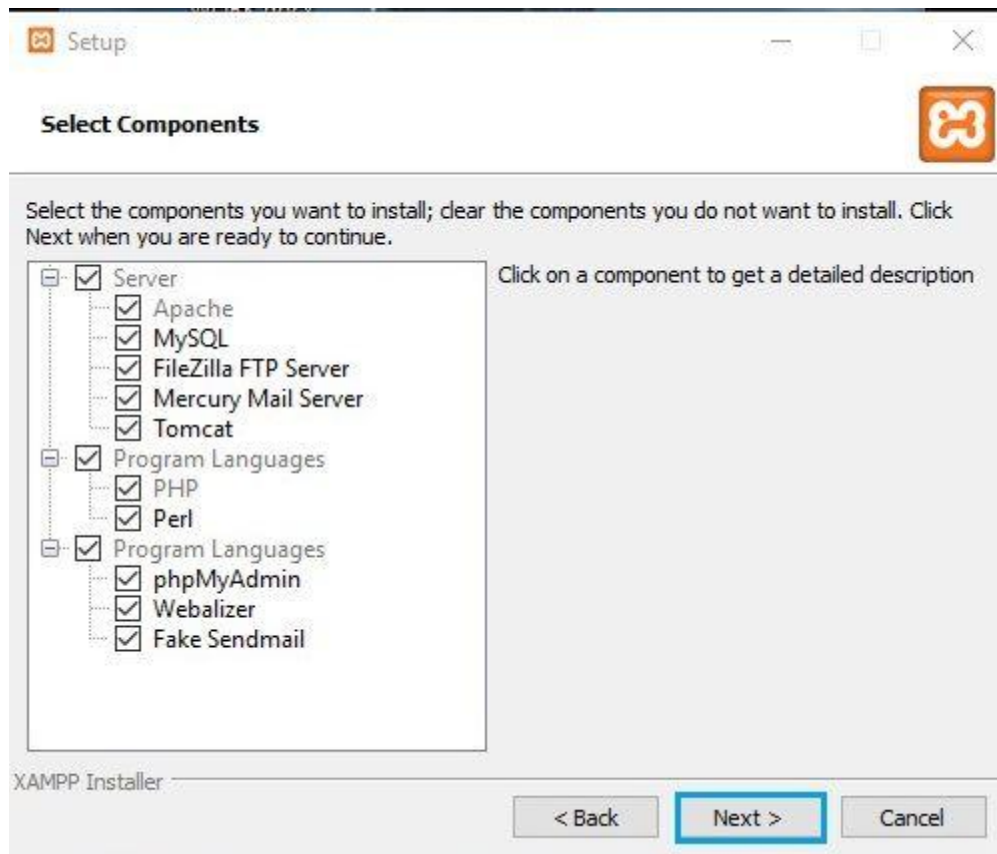
When its done downloading, go to your downloads, right click on the setup file and select "run as administrator".

This will take you to the **Setup-xampp** wizard:



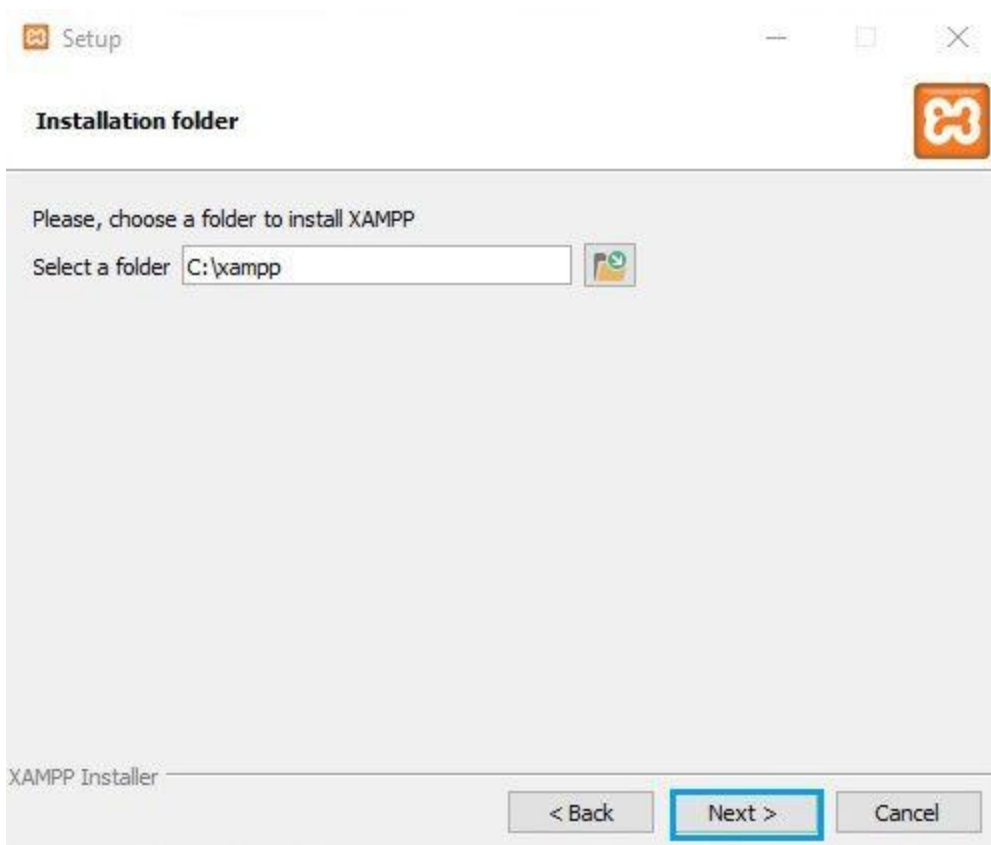
click next

Click next, and you'll be able to select the components you want:



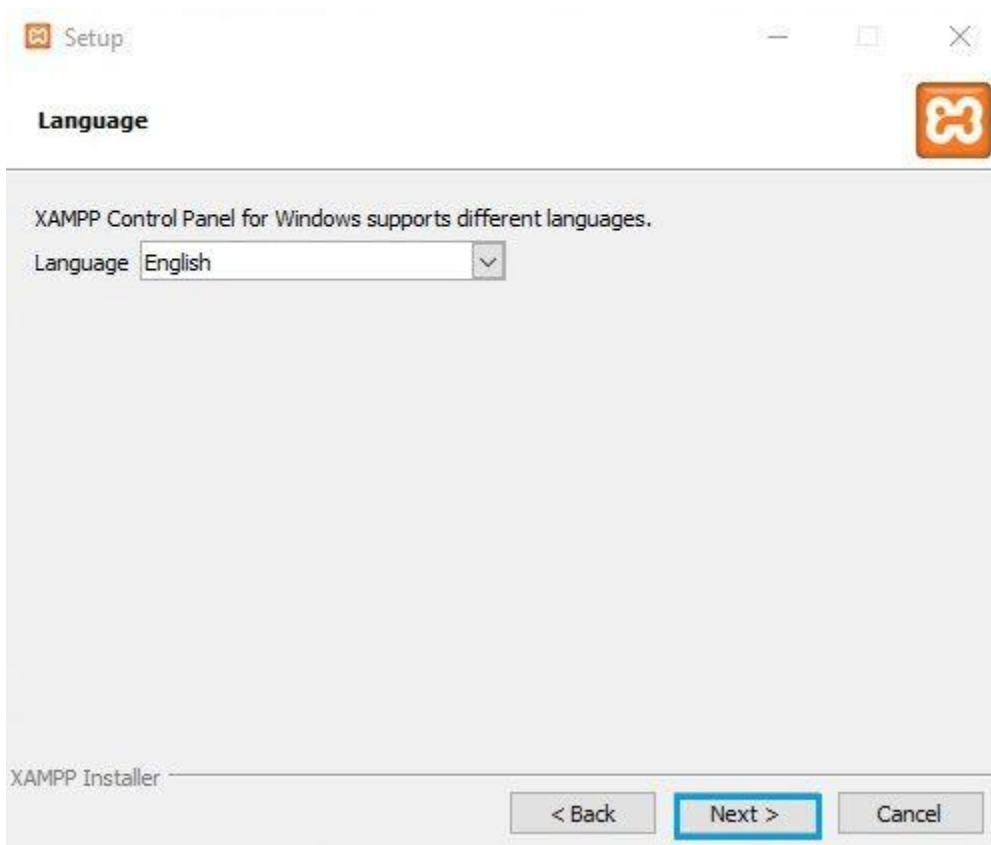
Select components and click next

Then you'll come to the installation folder. You must select a folder where you want to install XAMPP. I recommend creating a folder in your local disk to install XAMPP.



select an installation folder

Then you'll select the language. You can select either English or Deutsch (your choice):



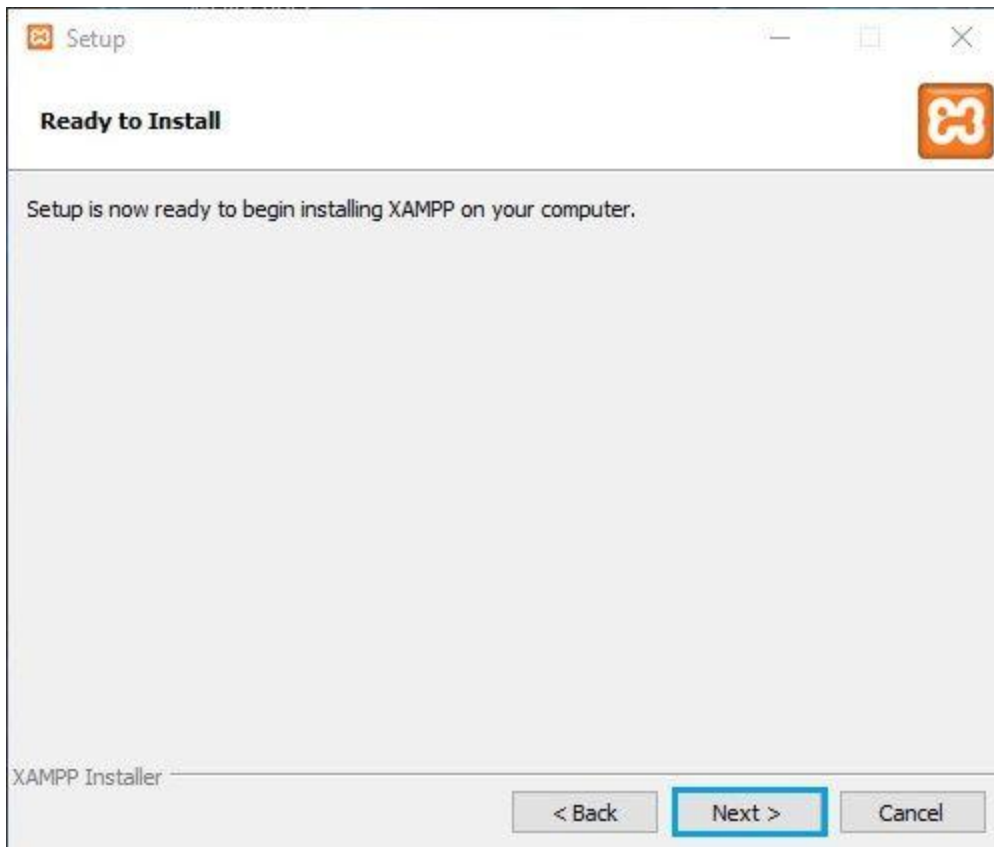
select language

Now you'll get Bitnami for XAMPP:



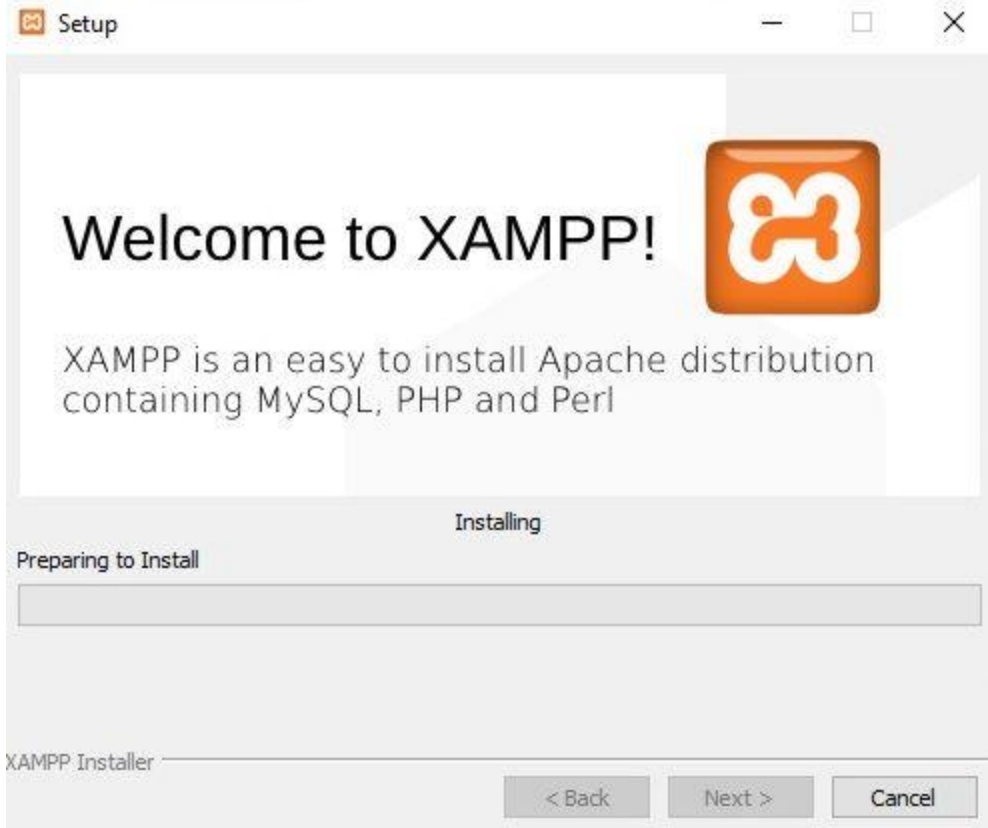
click next

And you're ready to install:



click next

Be patient while the installation process completes. When it's, done click ok.



wait for the installation to complete

Once the installation process is finished, you can now use XAMPP in your project.

Why Do You Need XAMPP?

To run PHP for the web, you will need to install a web server like Apache and a database like MySQL – and both are supported by XAMPP.

XAMPP is a local server that can run smoothly on our personal computer and is accepted in both Windows and Linux. It also helps you test websites and see if they work before publishing them to a web server.

How to run PHP with XAMPP

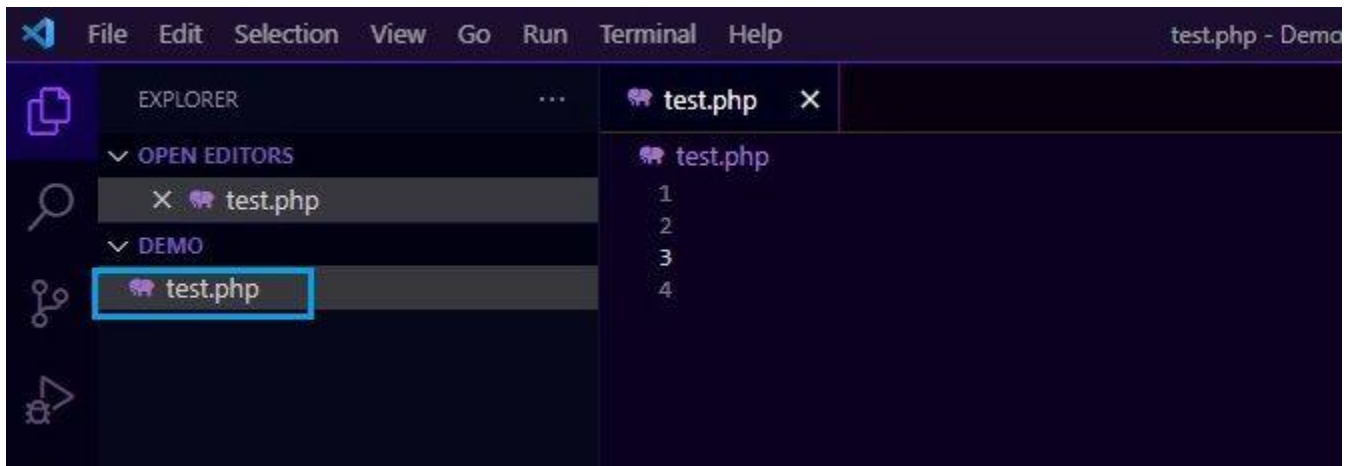
To run PHP with XAMPP you will have to go through some steps, and I will break it down so you can understand.

First, open the local storage folder, go to the “xampp” folder and open it. You should see a folder named “htdocs”. Open it then create a new folder in it. In my case I named the folder I created “Demo” (so give your folder the name of your choice).



Next, open your VS code, click on open folder, then go to the location where you saved the folder you created (which in my case I named “Demo”). Create a file with the

extension .php – in my case I named mine test.php. The extension .php tells the code editor that we are working on a PHP code/project.



create a file with an extension of .php

PHP is run with the `<?php` (Code goes in here) `?>` tag. The opening tag is `<?php` then your PHP code goes next before the closing tag `?>`. For example:

`<?php`

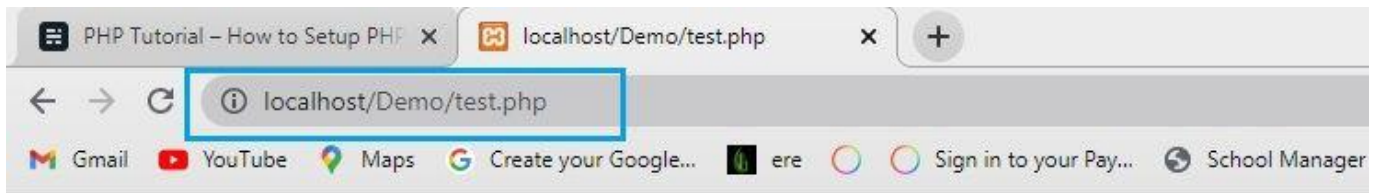
`echo "<h1> My Name is Derek </h2>;`

`?>`

The `echo` keyword tells the browser to display My Name is Derek while the `<h1></h1>` tells the web browser to format the text to be bold/bigger. Then save it.

After writing the code, open the XAMPP control panel, and start the Apache module by clicking on start under the action section.

Then go to your web browser and in the search bar type `localhost/Demo/test.php`, then enter. Your web browser should be display this:



My name is Derek Emmanuel

web display

If your code was displayed on the web browser, congratulations! You're up and running.

References

Tatroe, K. & MacIntyre, P., 2020. Programming PHP: Creating Dynamic Web Pages. 4th ed. Sebastopol, CA: O'Reilly Media.

Welling, L. & Thomson, L., 2017. PHP and MySQL Web Development. 5th ed. Boston: Addison-Wesley.

Nixon, R., 2018. Learning PHP, MySQL & JavaScript. 5th ed. Sebastopol, CA: O'Reilly Media.

Forta, B., 2018. SQL in 10 Minutes, Sams Teach Yourself. 4th ed. Indianapolis, IN: Sams Publishing.

Internet Programming as a Prerequisite for Advanced Programming Modules

Internet Programming 621/622 is a prerequisite for Programming 631/632 (Java - Diploma) and Programming 741/742 (C# - BSc). This ensures that students first gain experience in web-based programming before advancing to more complex programming concepts in Java or C#.

Internet Programming 621/622 (PHP & Web Development)

This module introduces students to server-side scripting with PHP, database interactions using MySQL, session management, security practices, and dynamic web application development. The skills learned here are crucial for understanding backend logic, data

processing, and web-based software development, which form the foundation for more advanced programming languages like Java and C#.

Why is Internet Programming a Prerequisite?

Web Development Knowledge – Understanding how web applications work is crucial before handling full-stack or enterprise-level programming.

Database Management – Teaches students how to interact with databases, a fundamental skill for both Java (JDBC) and C# (ADO.NET/Entity Framework).

Logical Thinking & Problem-Solving – Develops coding skills necessary for tackling more complex programming structures in Java and C#.

Backend Development Foundation – Before moving into Java desktop applications or C# enterprise systems, students learn the basics of server-side programming through PHP.

By completing Internet Programming 621/622 first, students are well-prepared to take on Java (Programming 631/632) or C# (Programming 741/742) and advance their skills in software development.